Hochschule Kaiserslautern
University of Applied Sciences

Angewandte Ingenieurwissenschaften Kaiserslautern

SYSTEM LEVEL RAPID DEVELOPMENT IN MECHATRONICS

WS 2024/25

TWO-WHEEL ROBOT PROJECT REPORT 1,2 and 3

SUBMITTED BY – Ishant Ishant

MATRIKEL. NR. – 889150

COURSE – MASTERS IN MECHANICAL ENGINEERING AND MECHATRONICS WS2024/25

UNIVERSITY – HOCHSCHULE KAISERSLAUTERN

Table of Contents

List of Figures

1. Abstract

This report presents the initial development of a two-wheel self-balancing robot based on the inverted-pendulum principle. MATLAB/Simulink and Simscape Multibody were used to construct a simplified mechanical model and investigate its dynamic behaviour. The model was subjected to external excitation and its position, velocity and angular responses were recorded.

A state-space feedback controller was developed to maintain the pendulum close to the upright position while following a commanded base velocity. The four feedback states were base position, base velocity, pendulum angle and pendulum angular velocity. The controller was first tuned manually and was then optimised using Control System Designer and Control System Tuner.

The final simulation stage produced a faster closed-loop response with zero overshoot for the selected tuning configuration. The nonlinear Simscape model was subsequently linearised using Model Linearizer to obtain a state-space representation for further analysis and controller development.

2. Multi-Body Simulation

2.1 Model Building

MATLAB 2023b and Simulink were used as the primary development environment for the multi-body simulation. The required Simscape Multibody and control-design toolboxes were configured before constructing the inverted-pendulum model.

The simplified system consists of a base body and a pendulum body connected through mechanical joints. The following dimensions were used:

Base brick: 0.05 m × 0.05 m × 0.05 m

Pendulum brick: 0.30 m × 0.02 m × 0.02 m

The base was connected to the World frame using a Planar Joint. An additional frame was created on the bottom surface of the base, and Rigid Transform blocks were used to establish the required orientation.

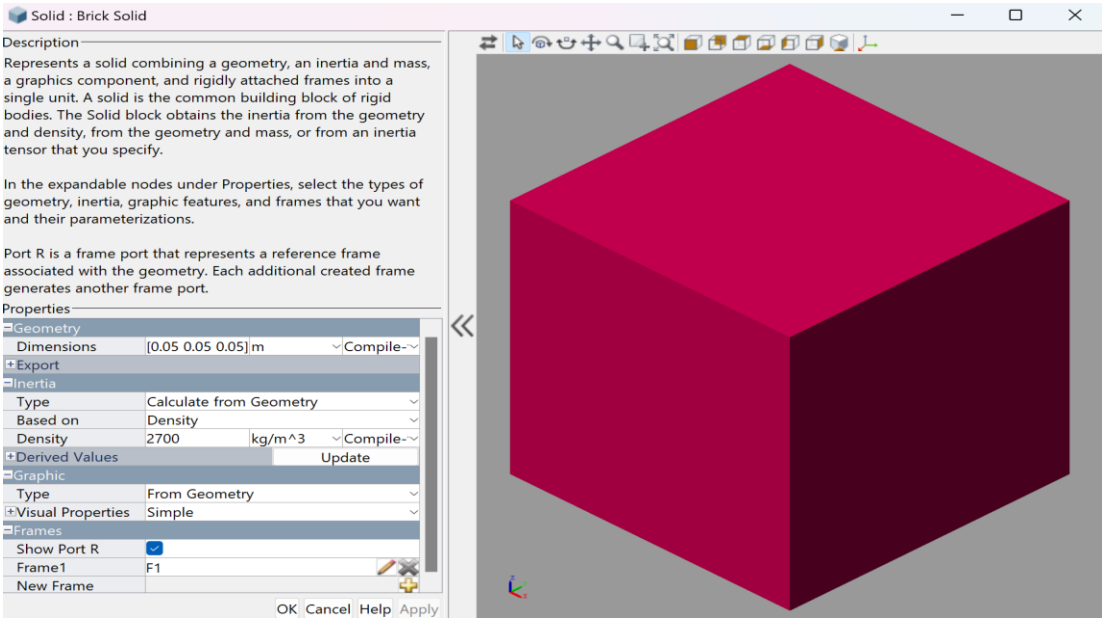

Figure 1 Base brick with dimensions and physical parameters

The pendulum was connected to the base using a Revolute Joint positioned 0.02 m above the centre of mass of the base. Further Rigid Transform blocks were used to position and orient the pendulum frame.

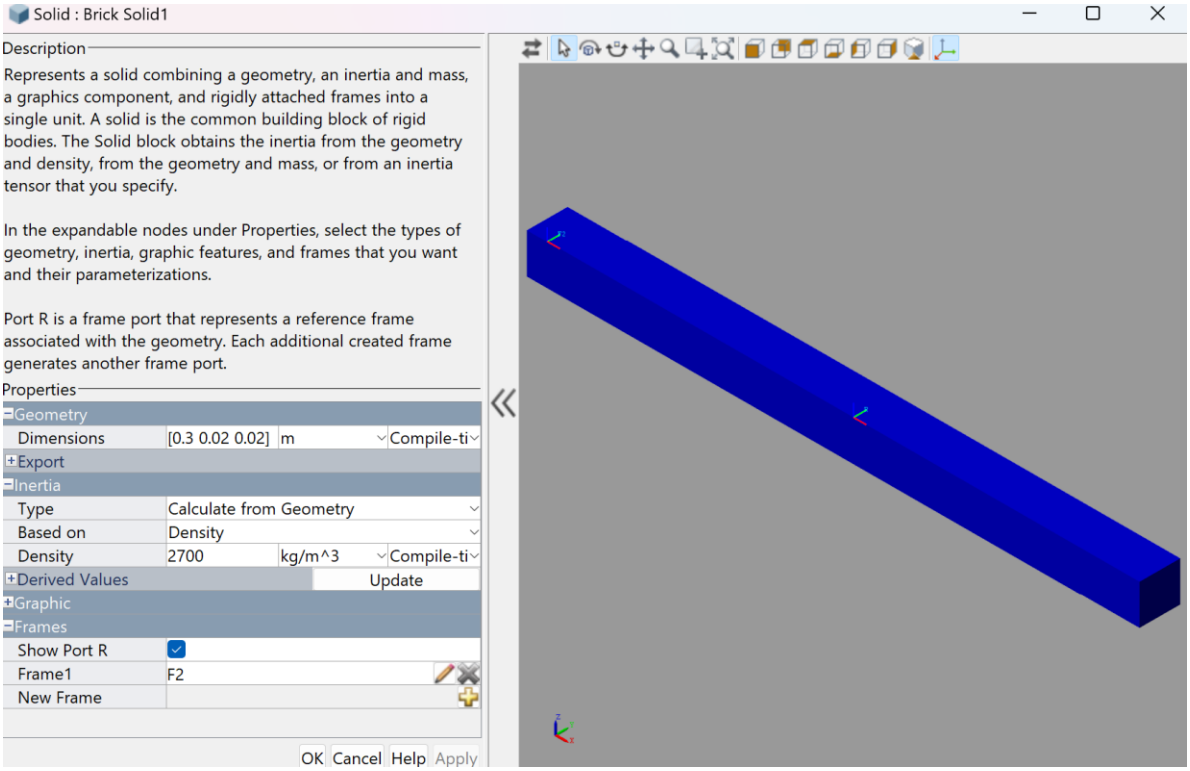

Figure 2 Pendulum brick and revolute-joint arrangement

The joint frames were configured so that the pendulum starts vertically at 0°. A −5° position target was then applied to introduce an off-balance condition and initiate rotation in the global Z–X plane. Masks and subsystems were created to simplify the model structure.

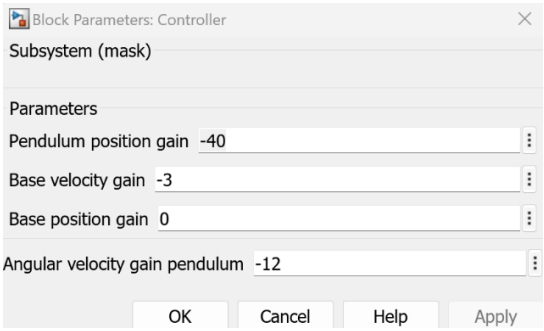

Figure 3 Masks and subsystems

A fixed-step solver with a step size of 0.001 s was selected for the simulation

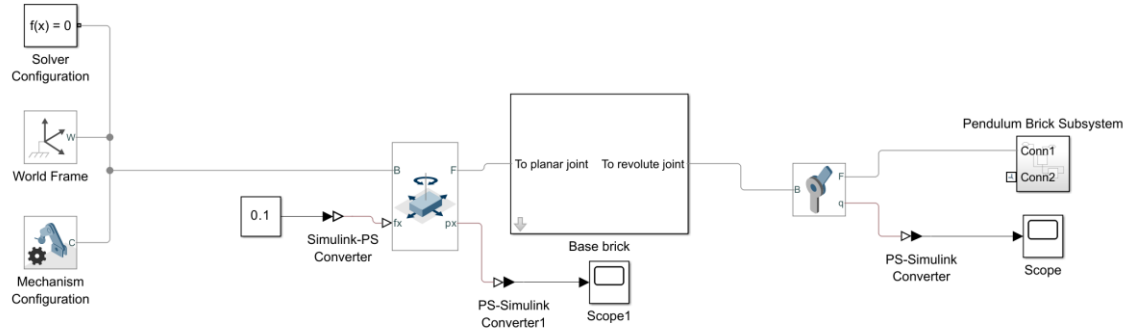

Figure 4 Complete inverted-pendulum model with base-force excitation

Video 1 – Initial model simulation
https://drive.google.com/file/d/1b9MHk0Aw2nNTITYWIaZtS4IFFunYR4PL/view?usp=drive_link

2.2 Data Logging and Actuation

Sensing outputs were enabled at both joints to observe the mechanical response. Base position and base velocity were extracted from the base joint, while pendulum angular position and angular velocity were obtained from the Revolute Joint. The signals were logged in Simulink Data Inspector and observed through Scope blocks using PS-Simulink converters.

Base position, X

Base velocity, $\dot{X}$

Pendulum angular position, θ

Pendulum angular velocity, $\dot{\theta}$

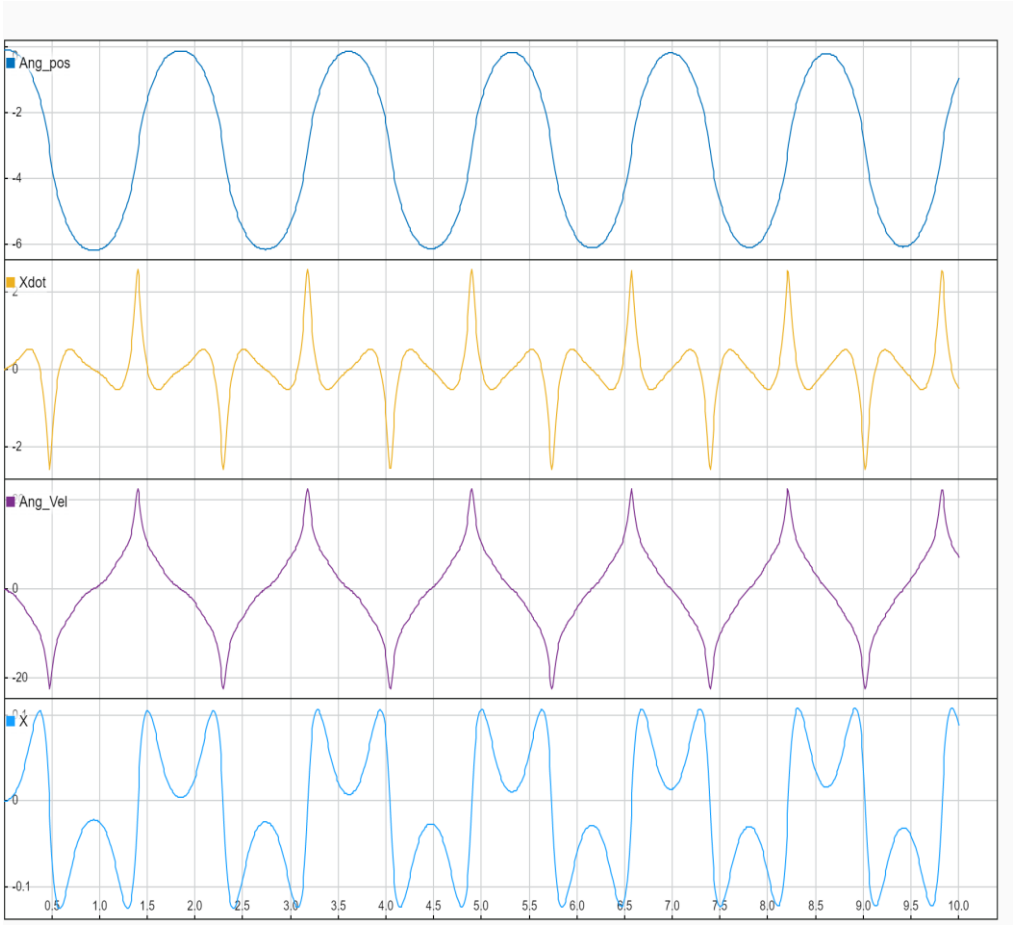

Figure 5 State-variable response without external excitation

An external excitation of 0.01 N was applied to the base in the X direction. A Simulink-PS Converter converted the Simulink command into a physical force signal in Newtons.

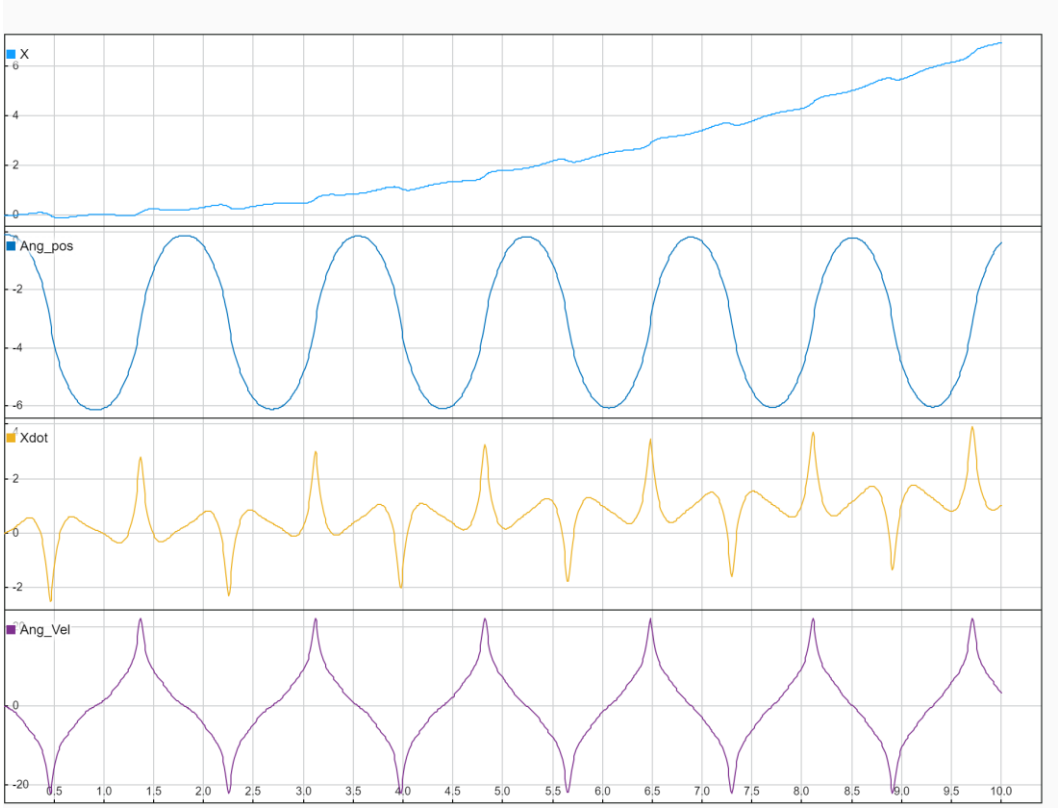

Figure 6 State-variable response with 0.01 N base-force excitation

The applied force changed the base motion to continuous movement in the direction of the force. Reversing the force also reversed the base motion, confirming the direct relationship between actuation and base movement.

3 Control Design

3.1 Model Set-up

For controller development, the Planar Joint was replaced by a Prismatic Joint so that the base had one translational degree of freedom along the global X direction. A Rigid Transform was added between the World frame and the base frame to establish the required orientation.

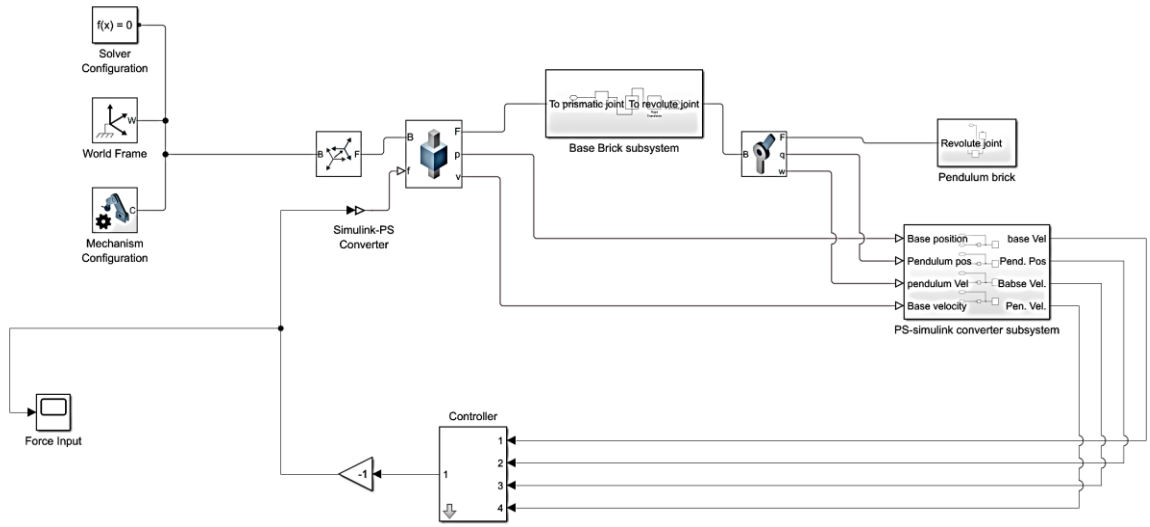

Figure 7 Inverted-pendulum model for self-balancing and controller design

Feedback was implemented for the four measured state variables. Each state was multiplied by an individual gain and the resulting signals were summed and negated to generate the force command for the Prismatic Joint.

Base velocity

Base position

Pendulum angular position

Pendulum angular velocity

Input Force = $-(K_v \cdot \dot{X} + K_x \cdot X + K\theta \cdot \theta + K\omega \cdot \dot{\theta})$

3.2 Manual Controller Set-up

The initial control objective was to reach the balancing equilibrium in the shortest practical time while minimising base movement. The four gains were adjusted manually. Initially, all gains were set to zero and the base-velocity gain was introduced first. A negative base-velocity gain produced the required stabilising behaviour, after which the other gains were adjusted.

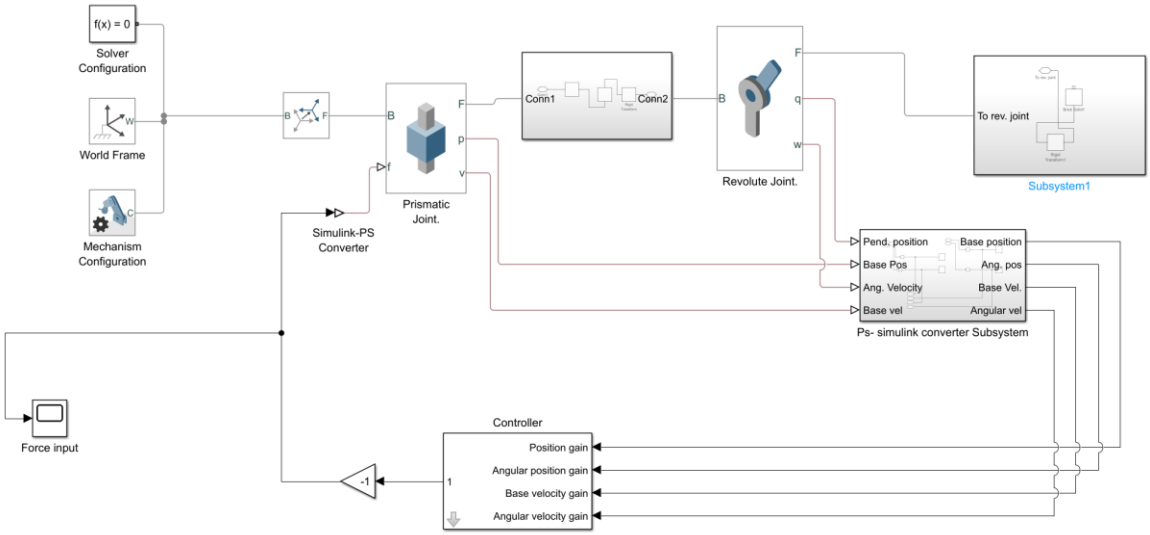

Figure 8 Manual gain configuration and balancing response

Video 2 – Self-balancing pendulum
https://drive.google.com/file/d/14HUWlODjr__v8sk9czE3MGhFKJgBQloQ/view?usp=drive_link

A fixed-step solver with a 0.001 s time step was retained. Compared with 0.05 s, the smaller step produced a smoother transition and faster system response.

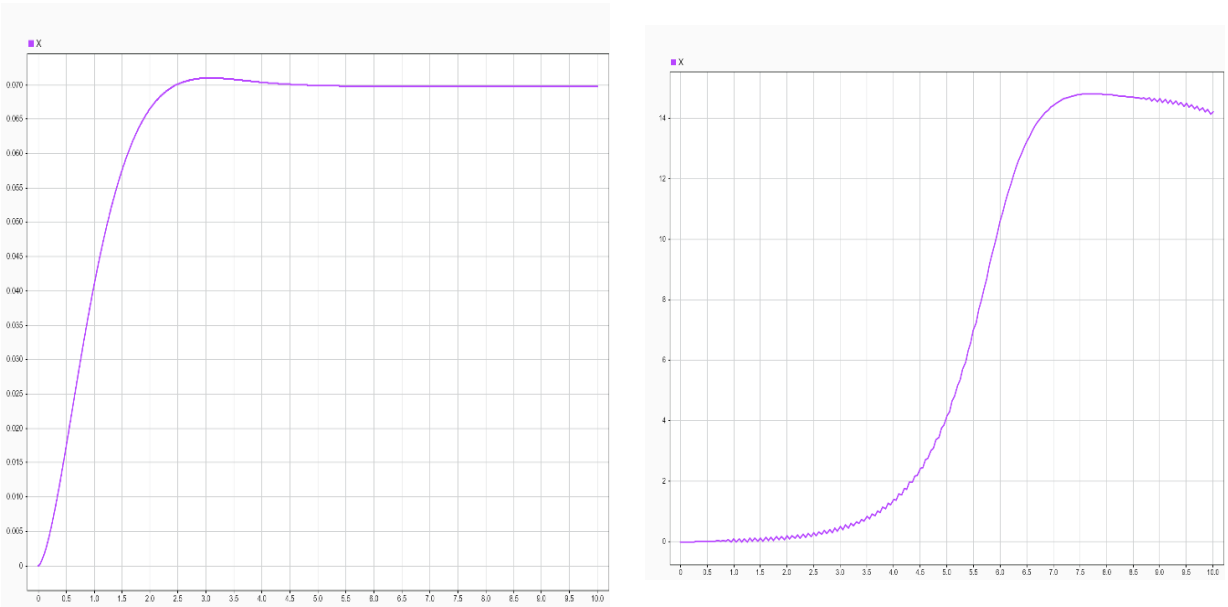

Figure 9 Base-position response for 0.001 s and 0.05 s solver steps

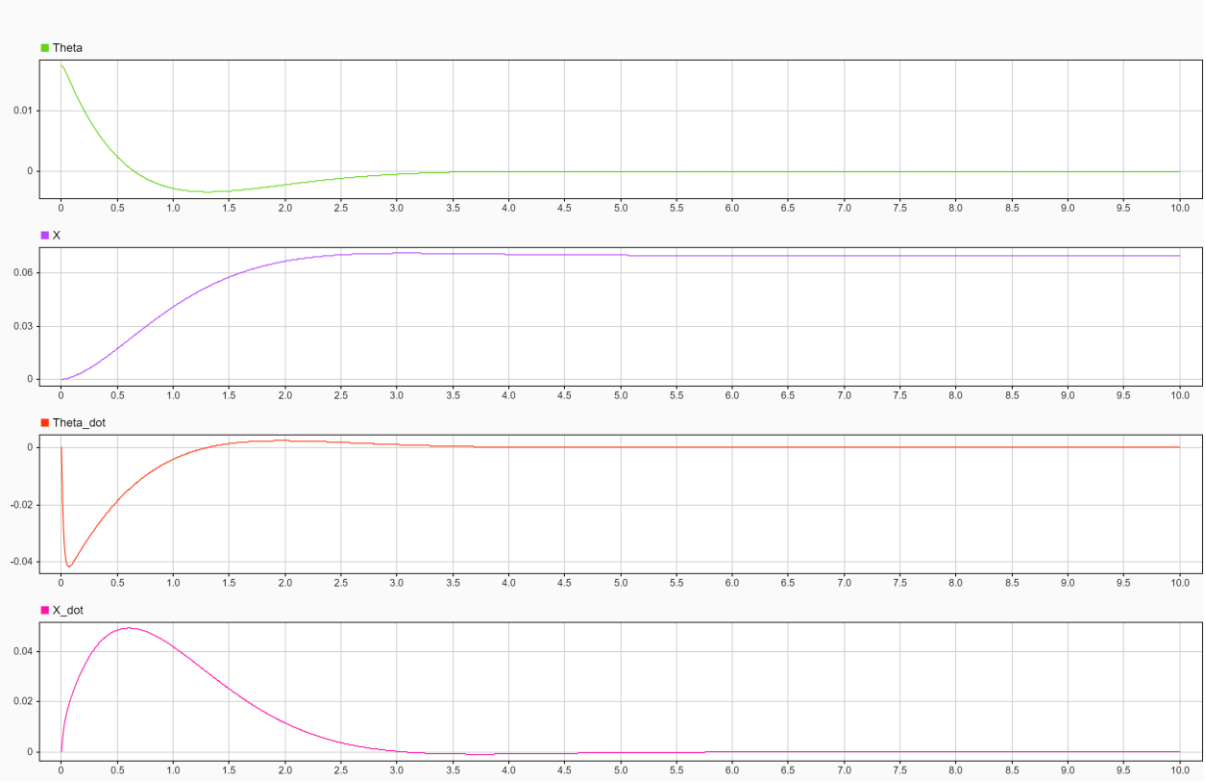

Figure 10 System state variables at 0.001 s

3.3 Target Base Velocity

A target base velocity was introduced using a Pulse Generator. The difference between the commanded target velocity and the measured base velocity was fed back to the controller.

Amplitude = 0.2

Period = 4 s

Pulse width = 50%

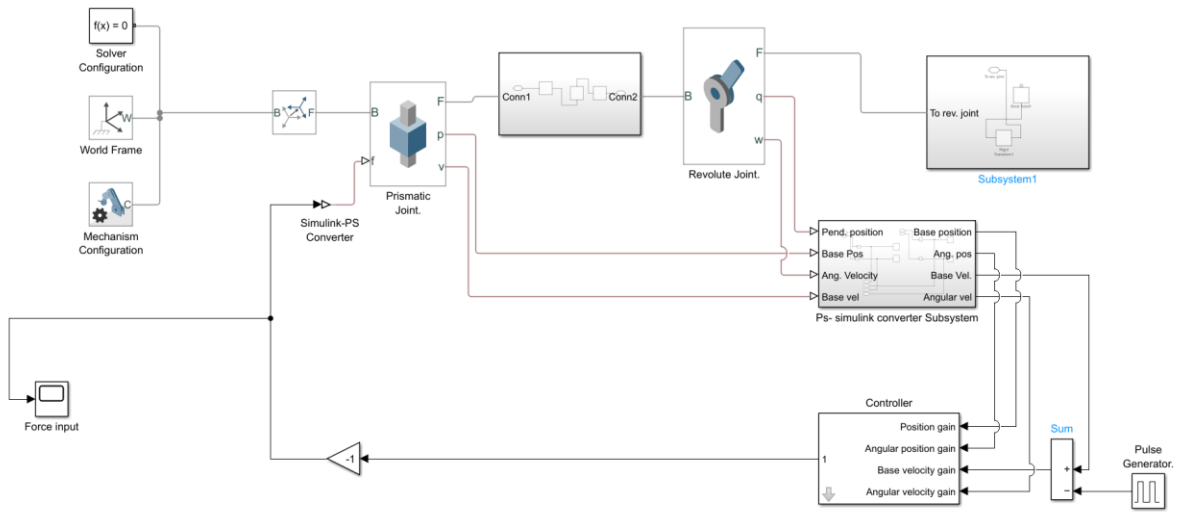

Figure 11 Inverted-pendulum model with target base velocity

Video 3 – Simulation with target velocity
https://drive.google.com/file/d/1REs9nGSpUF9BTxzF-RTeb3AQV64krlMT/view?usp=drive_link

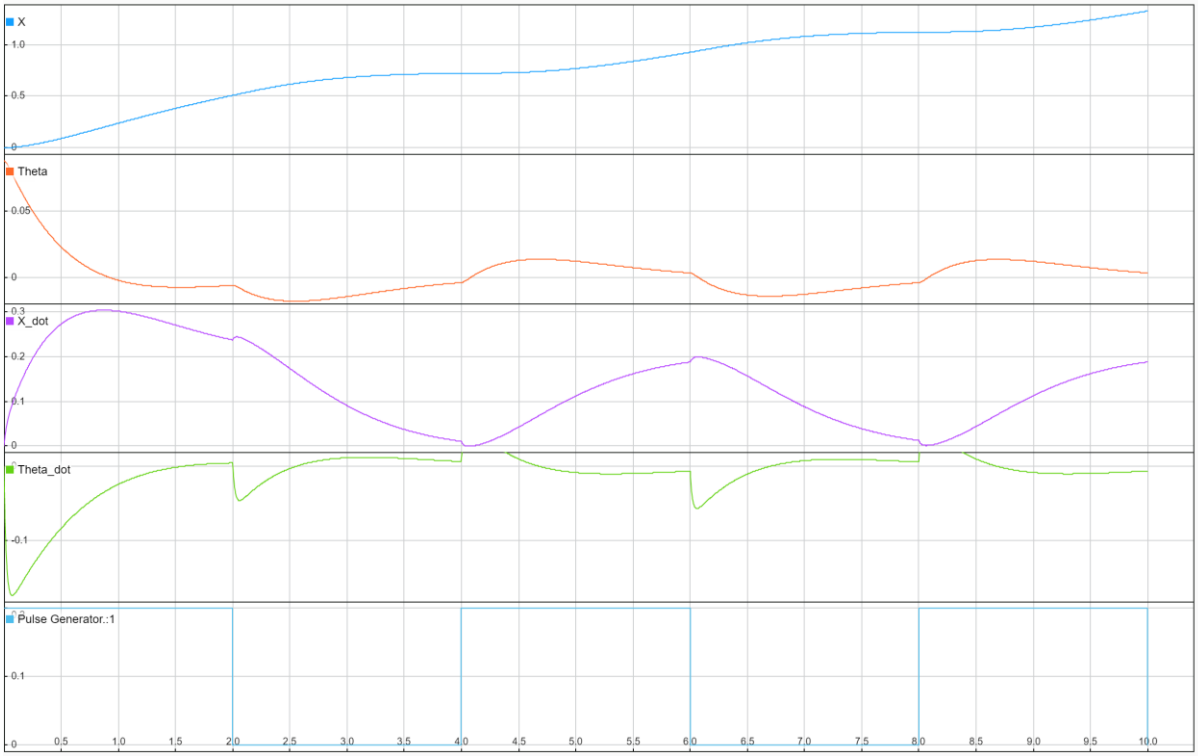

Figure 12 State variables during target-velocity simulation

The manually obtained gains for the target-velocity controller were:

Base velocity gain = −3

Base position gain = 0

Pendulum angular-position gain = −40

Pendulum angular-velocity gain = −11

The response remained stable but reacted too slowly to changes in the commanded velocity. Further controller optimisation was therefore required.

4 Optimisation of Control Parameters

The manual controller provided a stable starting point, but its target-velocity response did not satisfy the desired dynamic performance. The controller gains were therefore optimised systematically using the closed-loop step response.

4.1 Optimisation using Control System Designer

The base-velocity response and pendulum angular response were evaluated against the commanded target velocity. The following requirements were used:

Rise time < 1 s

Settling time < 2 s

Base-velocity overshoot < 5%

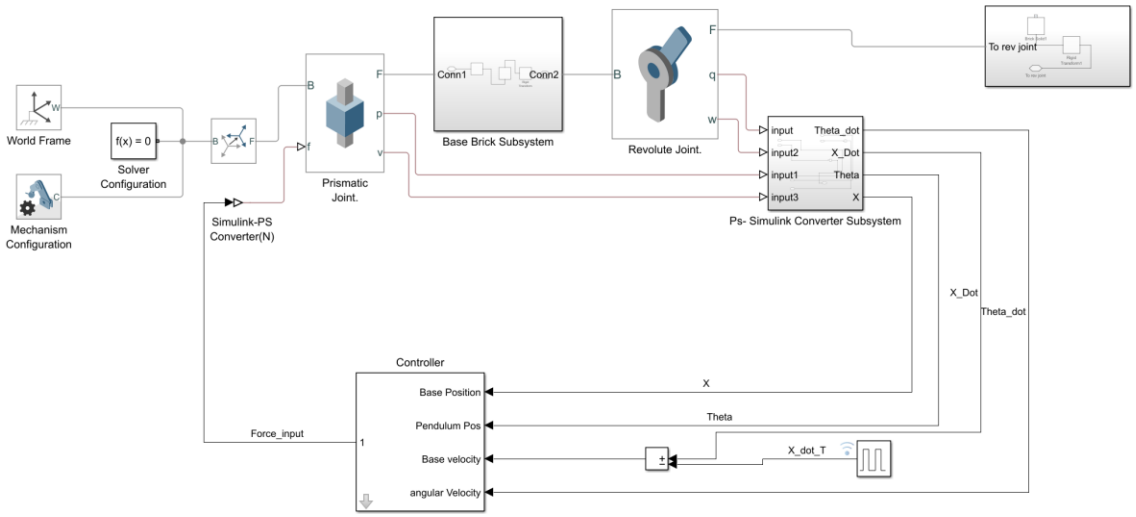

Figure 13 Control System Designer optimisation model

Several gain combinations were evaluated. Design 3 was selected as the most suitable configuration:

Base velocity gain = −12

Base position gain = 0

Pendulum angular-position gain = −50

Pendulum angular-velocity gain = −9

Figure 15 – Comparison of multiple gain iterations

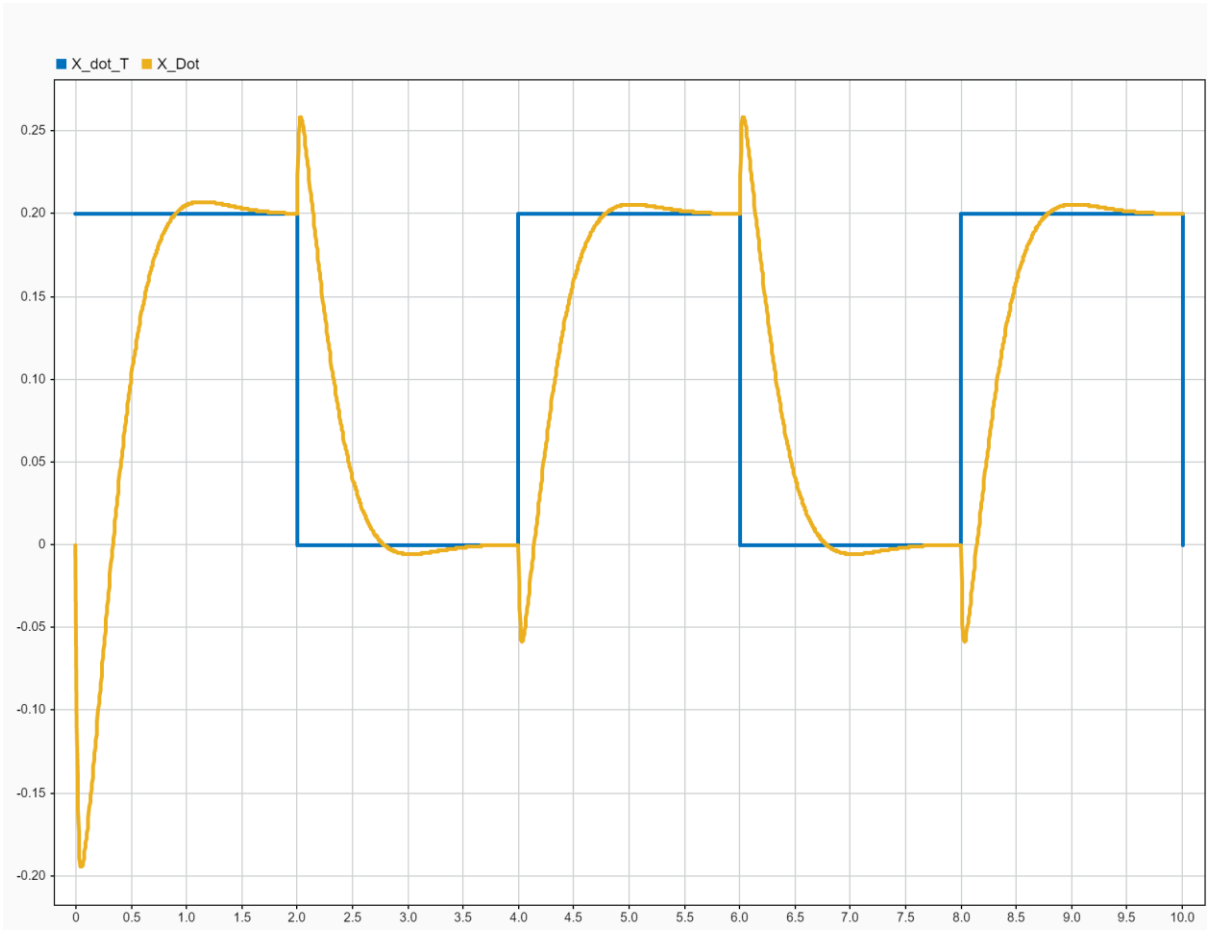

Figure 14 Target velocity versus output velocity with selected manual gains

The selected controller reached the target velocity in approximately 0.8 s with only a small overshoot, while the remaining response characteristics stayed within the defined design limits.

4.2 Optimisation using Control System Tuner

Control System Tuner was then used to automatically optimise the controller gains. The tunable gain blocks were selected, gain limits were defined and a step-command tracking goal was created between the target base velocity and measured base velocity.

Time constant = 0.2 s

Input = Pulse Generator target base velocity

Output = base velocity $\dot{X}$

Tuning goal = tracking of step commands

Figure 15 Target response with manual gains

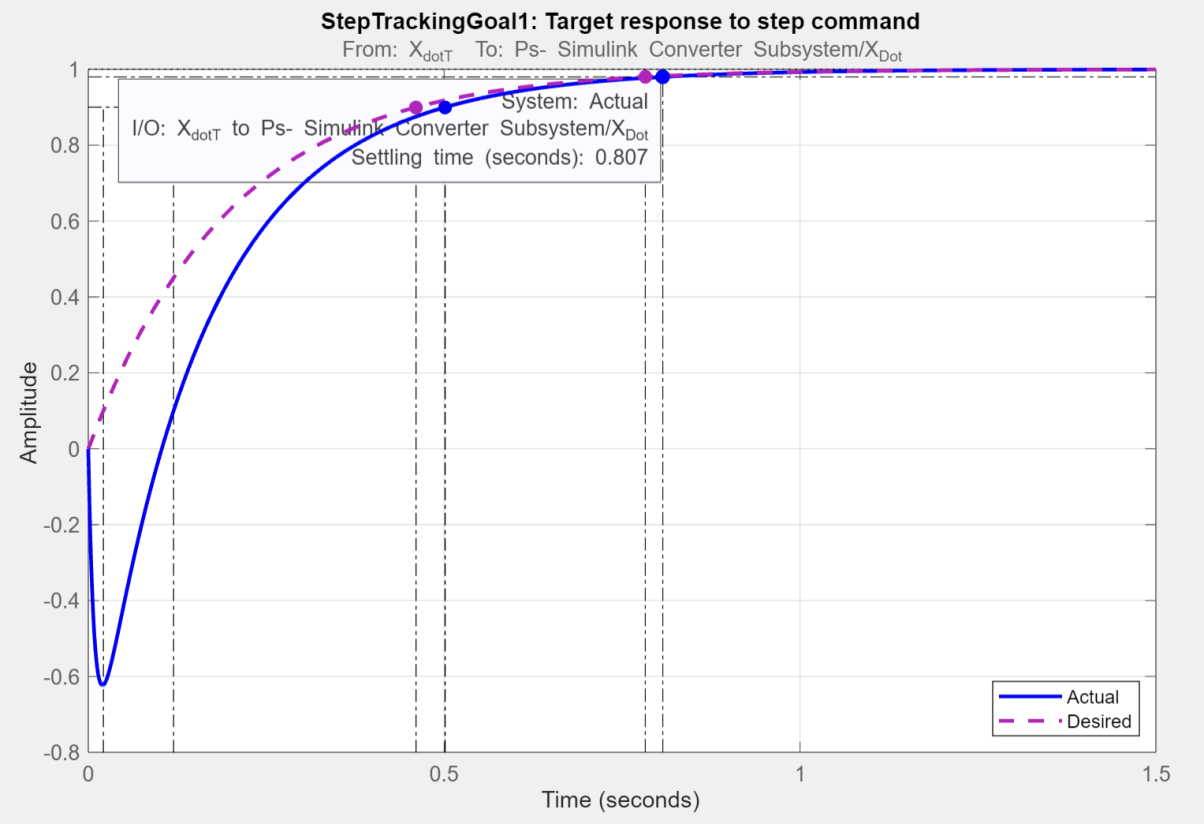

Figure 16 Target response with tuned gains

For a tuning time constant of 0.2 s, a settling time of 0.807 s was obtained compared with the desired settling time of 0.782 s. The tuned response showed zero overshoot and was more responsive than the manually tuned controller.

The resulting tuned gains were:

Base velocity gain = −41.89

Pendulum angular-velocity gain = −18.31

Pendulum angular-position gain = −150

Solver step size = 0.001 s

The higher gain values make the closed-loop response highly sensitive to changes in the input and state errors. The selected simulation therefore provided a fast response while maintaining the required balancing behaviour.

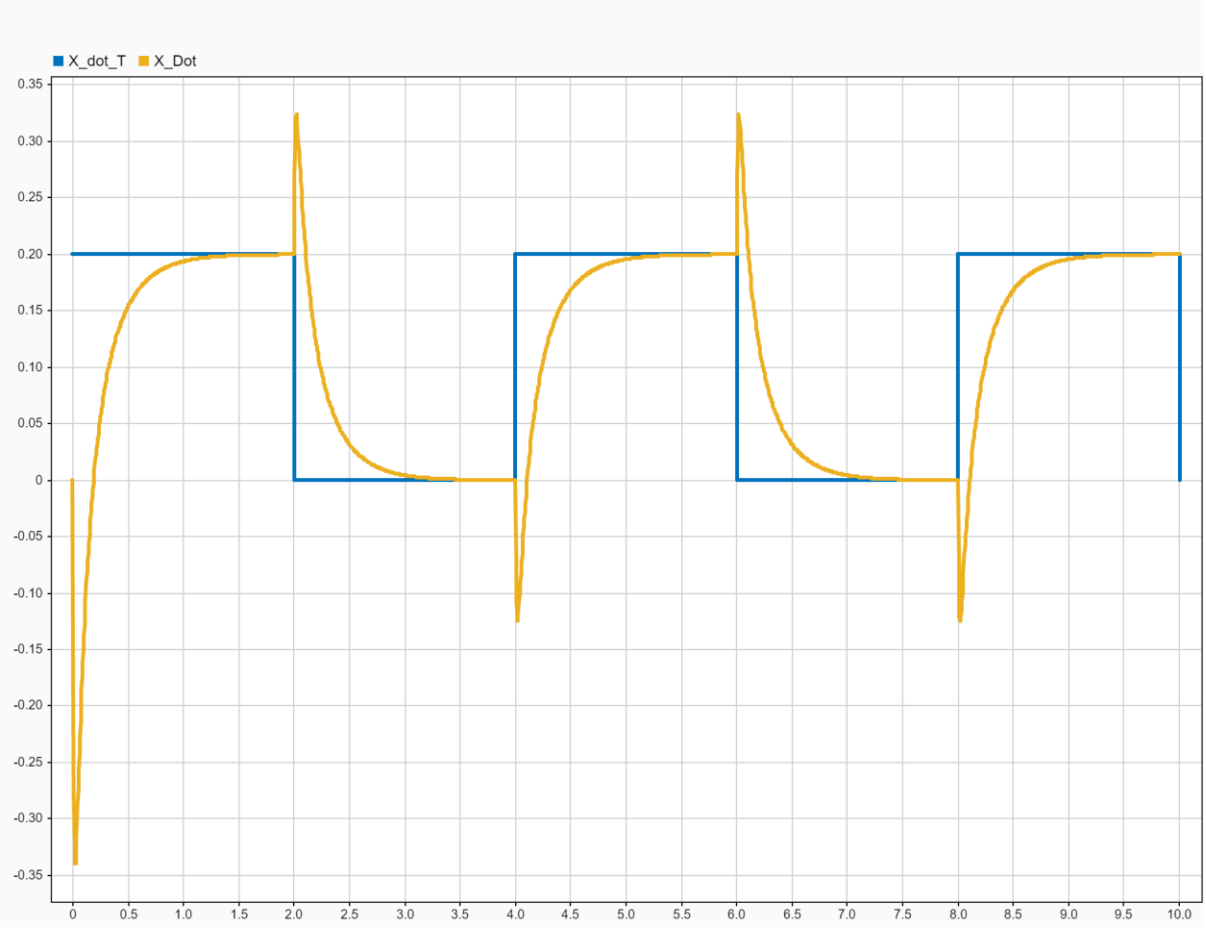

Figure 17 Final target-velocity response with tuned controller

Video 4 – Final tuned system response
https://drive.google.com/file/d/1dNuiezrtpew7L13N8F1UQUyWz_NyrC4y/view?usp=drive_link

4.3 Linearisation of the Simscape Model

The nonlinear Simscape model was linearised using the Model Linearizer application. The state ordering was configured so that the resulting input and output matrices followed a clear physical state representation.

State ordering:

x1 - inverted_pendulum_control_design_SS_Linearization_1.Revolute_Joint.Rz.q
x2 - inverted_pendulum_control_design_SS_Linearization_1.Prismatic_Joint.Pz.p
x3 - inverted_pendulum_control_design_SS_Linearization_1.Revolute_Joint.Rz.w
x4 - inverted_pendulum_control_design_SS_Linearization_1.Prismatic_Joint.Pz.v

y1 - Theta
y2 - X
y3 - Theta_dot
y4 – X_dot

A =

	x1	x2	x3	x4
x1	0	0	1	0
x2	0	0	0	1
x3	46.91	0	0	0
x4	-0.6459	0	0	0

```
B =
       u1
 x1     0
 x2     0
 x3  -4.955
 x4   0.964

C =
    x1  x2  x3  x4
 y1  1   0   0   0
 y2  0   1   0   0
 y3  0   0   1   0
 y4  0   0   0   1

D =
    u1
 y1  0
 y2  0
 y3  0
 y4  0
```


5 STATEFLOW AND STATE MACHINES

5.1 Stateflow Onramp course

Recently I completed the Stateflow Onramp course which helped me to build up the simple state machines like “Robotic Vacuum” and “Simple Traffic Signal Lights”. Further I applied the gained knowledge in our project.

Figure 18 Certificate On ramp

5.2 Simple State Chart – Toggle Switch

By using “after” command (time-controlled system) a simple state chart was created which toggles between ‘On’ states and ‘Off’ states every two seconds. Expected output result I got in scope block.

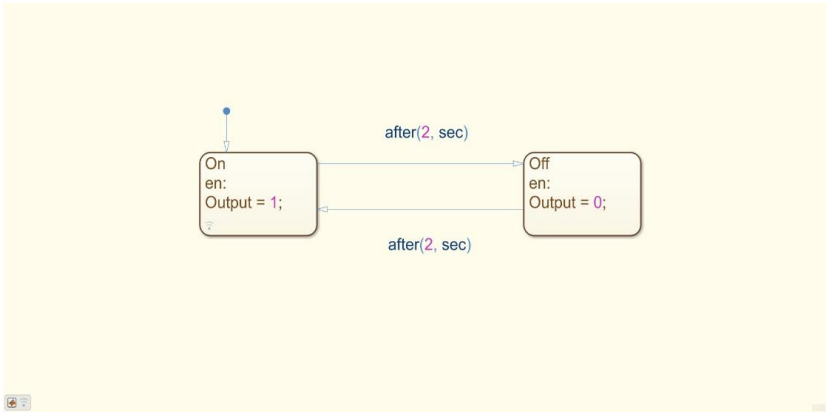

Figure 19 Simple toggle switch state chart

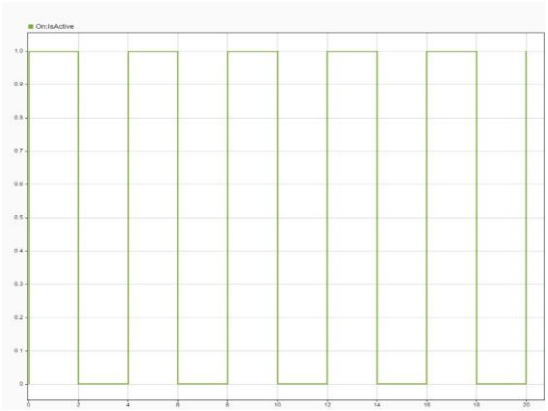

Figure 20 The output variable 'Toggle Status' from the Data Inspector of a simple toggle switch state chart

Video 5 - https://drive.google.com/file/d/1LYoFYbgRBEOxtUmcYyCWVM9n2cx-KpQ4/view?usp=drive_link

5.3 States chart with input values/ports

The state chart is elongated with an input port and along with a new state is created "Error" in which the system enters when the value is '0' and exit the value other than '0'. For excitation on the input port a impulse generator is used further.

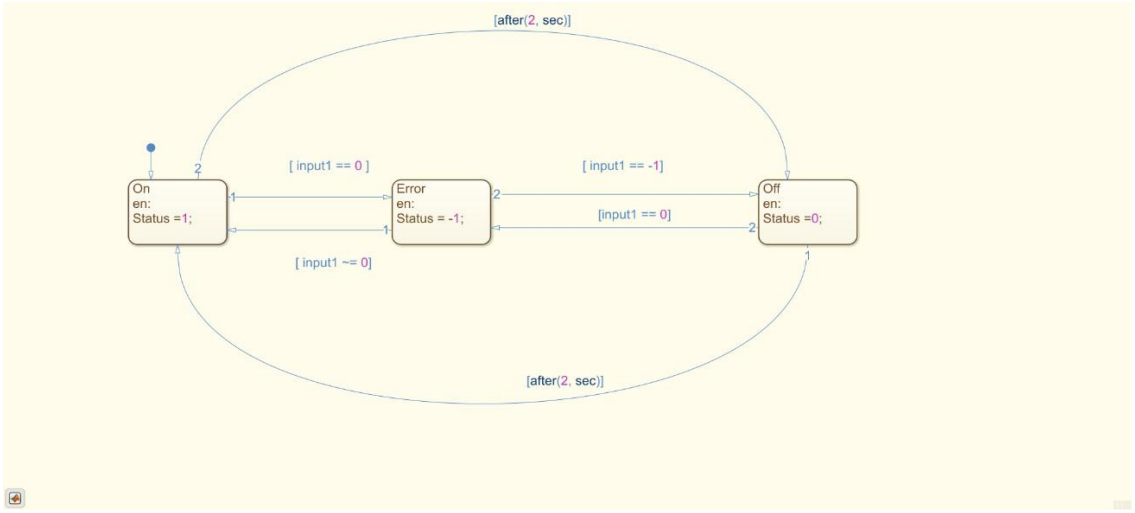

Figure 21 The state chart of the toggle system with an error

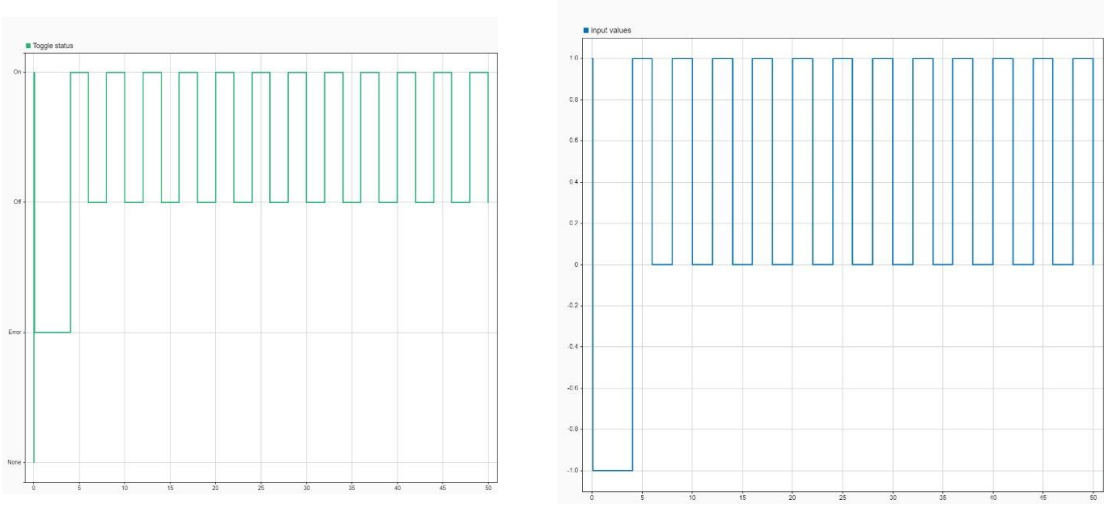

Figure 22 The input variable 'Input values' (blue) and output variable 'Toggle Status' (green) in time plots

Video 6 -
https://drive.google.com/file/d/1zNuwmeYFoQGzrIhaNo3Ur1dXg2nqYewW/view?usp=sharing

5.4 Truth Tables

The same toggle switch model was extended to receive inputs through a truth table. Therefore, a truth table block and a sine wave block were created. The sine wave block was used as the input to the truth with an amplitude of 1 rad/s. An input variable called 'tt_input' and an output variable called 'tt_output_flag' were created in the truth table. The truth table was set up to generate an output of '1' for an input greater than 0.5 and '-1' for inputs lesser than or equal to 0.5. The curves in the given collateral do not cross each other exactly at 0.5 because of the large solver's step-size. For a step-size of 0.1, the curves cross each other at 0.5.

Figure 23 The condition table (left) and action table (right) inside the truth table

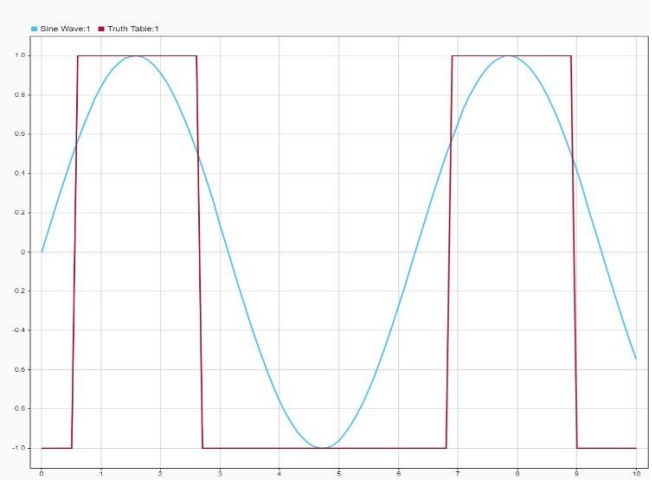

Figure 24 The output of the truth table (green) compared with the sine wave (orange)

5.5 Superstates and Solver step-size

Two parallel sub-states were created in a superstate. Basically, another sub-state was created to double the output of sine wave. In order to both sub-states were working simultaneously for every time step, the decomposition was set to the parallel. As both the sine wave was compared, I found that the sine wave had lower step size (0.01) is smooth than the sine wave had higher step size (0.2).

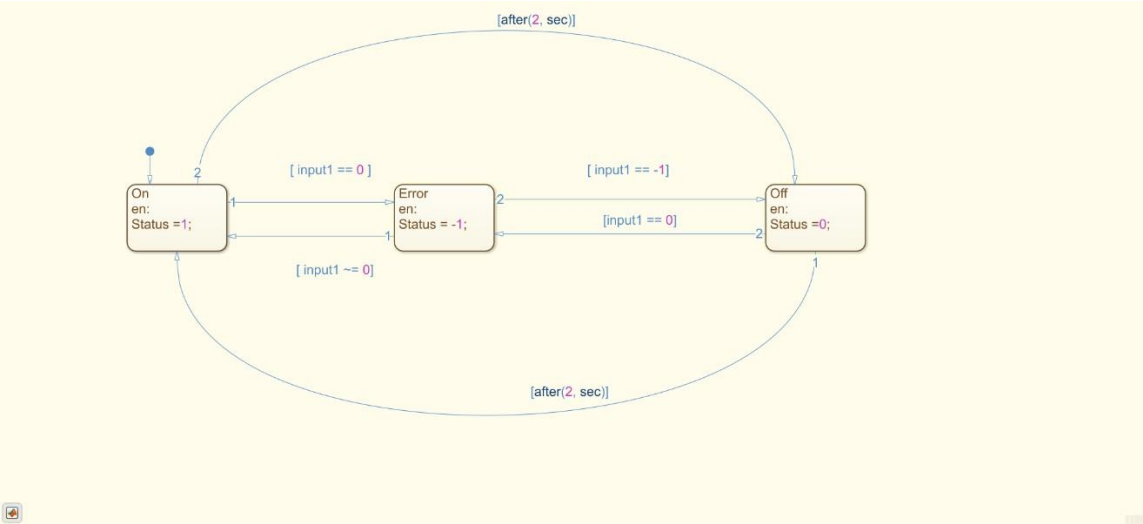

Figure 24 The state chart of the toggle switch with the superstate and its sub-states

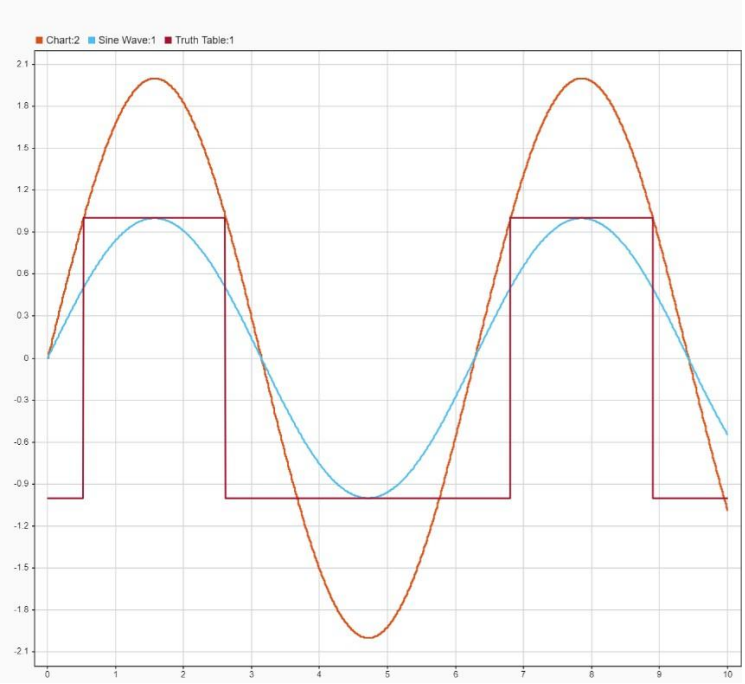

Figure 25 The plots of the sine wave (blue), the doubled sine wave (green) and the truth table (orange)

5.6 Simulink function in state chart

The Simulink function was dragged down from the toolbox directly into one of the parallel sub-states so that the input and output blocks was automatically created inside Simulink function block. A gain block with the value ‘2’ is added in the Simulink and connected between the input and output blocks.

Then the function is called in the second parallel sub-states to execute it during simulation. The graphs outputs were same as the “Superstates and Solver step-size”. Both the outputs are exactly same with different methods.

First, a line-based formula was coded into the state and in this method, a Simulink function was created and called into the state.

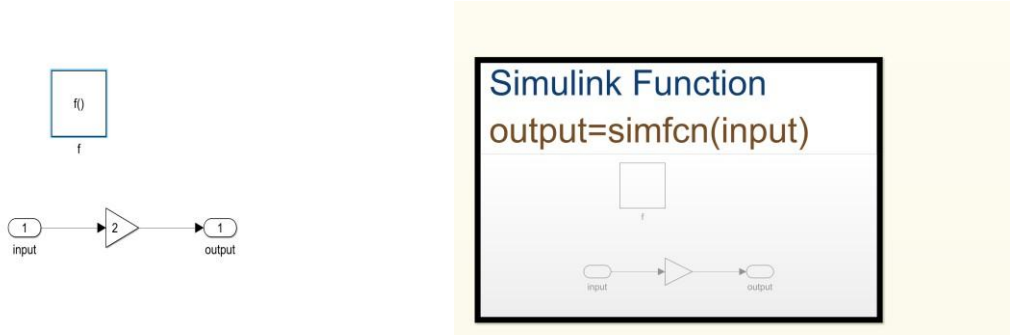

Figure 26 An image of the Simulink function used inside the state chart for doubling the sine wave

5.7 Triggered state charts

A event port “Trigger” is created to trigger the state charts at specific frequency. It was joined with function call generator which act as “trigger system”. Moreover, the sample time set to the 0.1 to represent the 10 Hz. The final result of doublesine wave was looked like staircase than the sine wave.

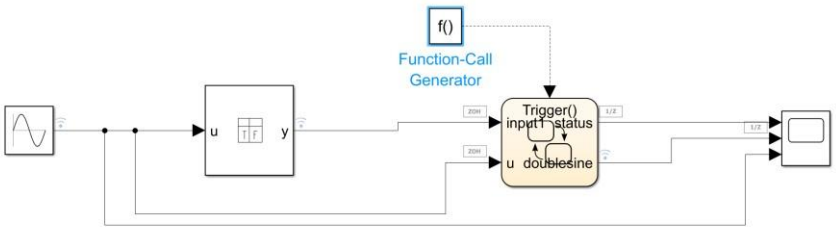

Figure 28 The toggle switch model with a triggered state chart using a function call generator block

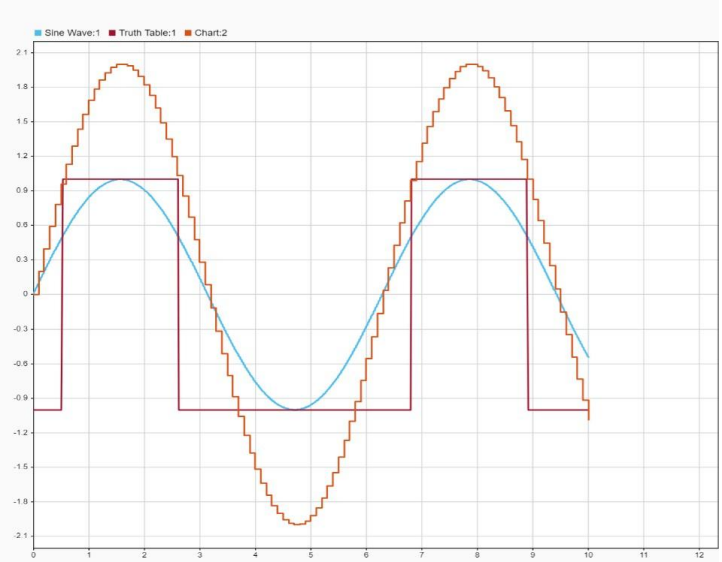

Figure 29 Plots of sine wave (orange), double sine (red), and truth table (purple) of the triggered-subsystem

6 Code Expert

6.1 Arduino IDE and SAMP boards

The Arduino IDE was installed to develop and simulate the algorithms in Simulink to run them on the device. In addition to that, SAMP board is proposed by the Arduino IDE to install for better functioning of Software.

6.2 Blinking Light

A sample model is developed by connecting the “Display Output” and “Pulse Generator”. Pin 13 and Pin 14 are excited with pulse, and they are output LEDs in Arduino board. Both were connected with the Model so the output was received as blinking of LEDs one by one as showed in video.

Figure 30 The fast blink test model on pin14 (left side) and the slow blink test model on pin 13(right-side)

Video 7 ---

https://drive.google.com/file/d/1MlIM4OHu9bzeWx_39n8OzPukXCol6IUQp/view?usp=drive_link

6.3 Reading Sensor Data

In order to check the accurate code deployment, the “Display Output” and “Display Input” are connected with pulse generator, constant and scope respectively. The output graph showed the presence of obstacles near the robot which can be watched on “Monitoring Mode.”

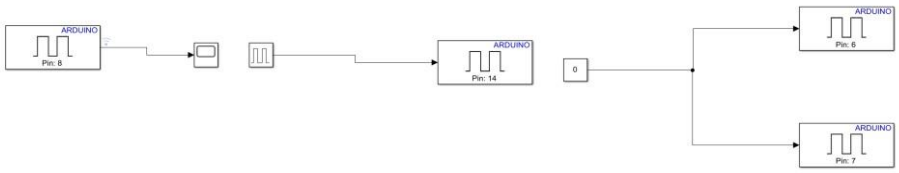

Figure 31 The Simulink model to read blink LED

Video 8
https://drive.google.com/file/d/1zNuwmeYFoQGzrlhaNo3Ur1dXg2nqYewW/view?usp=drive_link

6.4 Reading the IMU data using blocks from MecRoKa Library

The “IMU6000” block is dragged down in Simulink space to gain the sensor data. It was connected with “Mux” block at “simacc” and “Mux” block consisted of the three “int32” based vector datatype. The data type conversion is used to round off the value otherwise the resulted output is not an expected sine wave, it varies in between 0 and -1. To get the results the scope block is used which is connected with “Demux” block. When the model is connected with the cable the blinked LEDs are showed for confirmation.

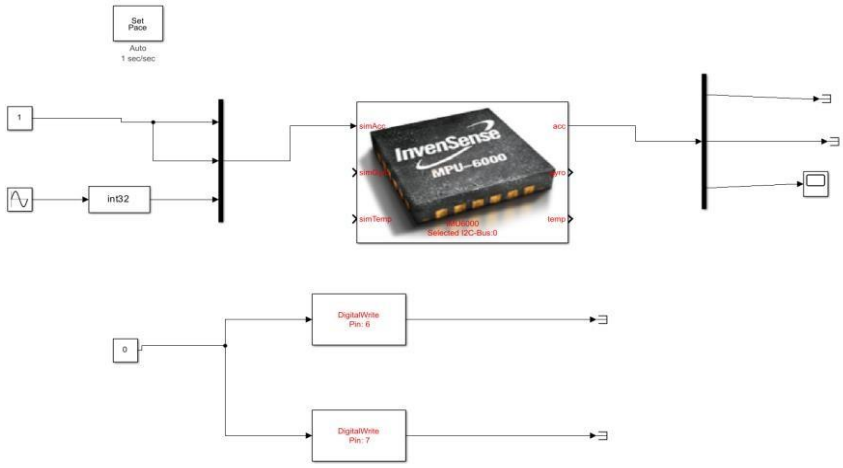

Figure 33 The Simulink model for reading the IMU 6000 data using the MecRoKa library

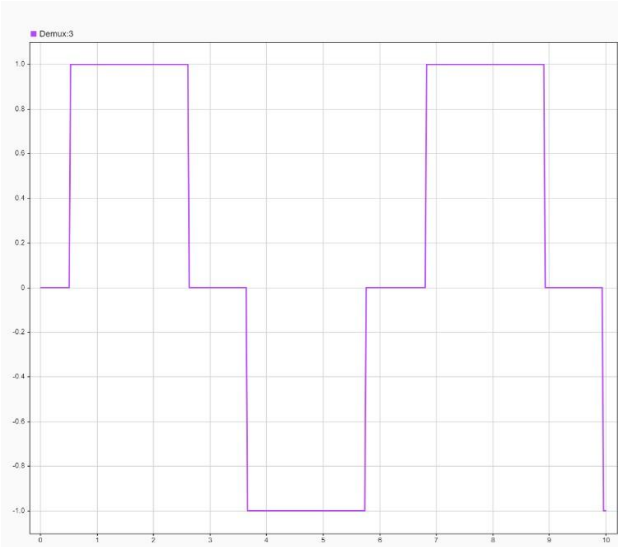

Figure 32 The output before demux depicting a sine wave converted to int32 data type

6.4 Setting up and testing the TWO-WHEELED ROBOT

6.4.1 The virtual Robot

The virtual robot was ran in Simulink model which uses the circular plate to show the robot body (Wheel and Axle) and IMU sensor.

Video 3

https://drive.google.com/file/d/1xRZK9UrYD55u0OPK7tx8MeDxFDuuQKkV/view?usp=drive_link

6.4.2Initial movement test of the robot

The Simulink model of MecRoKa is placed on the robot and further examined for movement and functioning. In addition to this the COM port is generally set up on the “Manually Select” which was changed to “Automatically Select” because the software couldn’t communicate with Arduino board without specifying it to COM port.

https://drive.google.com/file/d/1n-u6WEyPts0qtJ6lqGp5VqsglUpOhPgd/view?usp=sharing

First, few second was taken by robot to stabilize, then it turned 90 degree and accelerate until it was falled. Video 4
https://drive.google.com/file/d/1AzvBhZ4xQF2hkag15L43vjau29v--KVI/view?usp=drive_link

7 Challenge 1 – Driving As fast as possible 3 m

7.1 Improve feedback gains

The Velocity, pitch and pitch gain rate were applied to regulate the stability of the Two-Wheel Robot in the simulation model. The velocity gain is responsible for managing velocity control errors within the system. When this gain is increased, the system responds more aggressively to velocity control errors but may also result in overshooting and instability. Conversely, reducing the gain improves reliability and consistency but introduces steady-state errors and slows down the system's response. These effects were also observed during the real-world testing of the robot.

The pitch gain regulates the pitch control error, leading to significant oscillations when the gain is increased. On the other hand, reducing this gain causes the system to respond more slowly.

The pitch gain rate regulates the system's oscillations and response. Increasing the gain reduces oscillations but slows the system's response, while decreasing the gain has the opposite effect. It functions as a damper for the system.

As outlined in the guidance PDF, the controller feedback gain values were adjusted using the hit-and-trial method to stabilize the robot and enhance its system response.

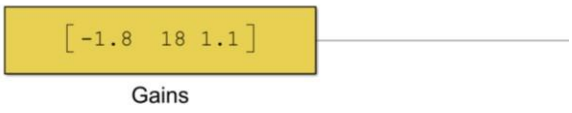

Figure 34 Feedback gains

7.2 Increase Integrator limit and BLDC step limit

The robot was unable to achieve a velocity exceeding 0.5 m/s due to the saturation limit of the ‘a_to_v_integrator’. To enable the robot to reach its maximum velocity and cross the 3-meter path in the shortest possible time, both the upper and lower saturation limits were increased. Additionally, the BLDC step limit also restricted the robot from attaining higher speeds. To address this, the upper and lower step limits were adjusted to +2 and -2, respectively. As a result, the device was able to operate beyond the previously set 0.5 m/s limit.

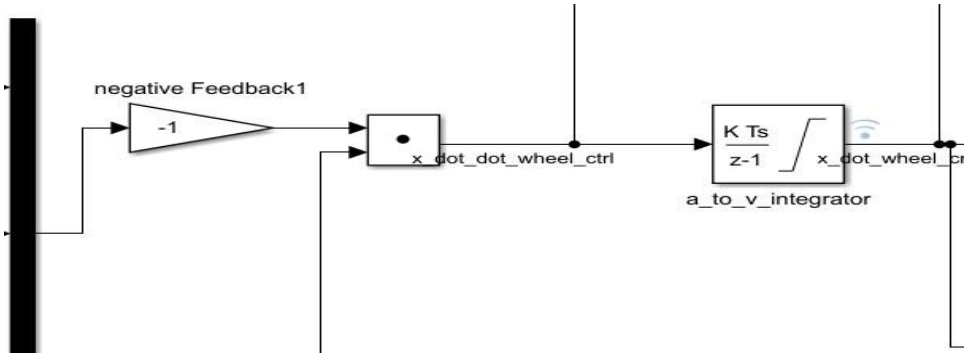

Figure 35 Integrator "a_to_v_integrator

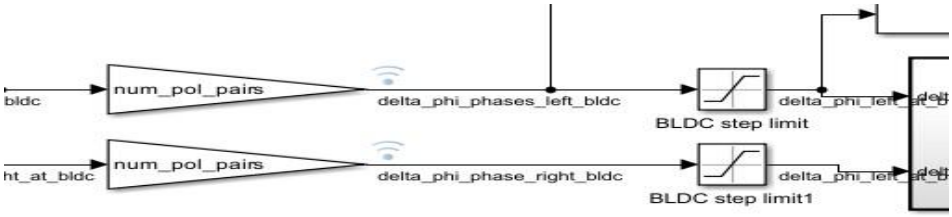

Figure 36 BLDC step limit saturation block

7.3 Gains Adjustment by HIT and Trail Method

The gains (velocity gain, pitch gain, and pitch rate gain) were adjusted multiple times to achieve the optimal solution. Ultimately, the gains were set to -1.8, 20, and 1, respectively, and deployed in the robot at a target velocity of 1.5 m/s. This allowed the device to reach the finish line in approximately 3 seconds without a run-up. With a run-up of 0.5 meters, it completed the 0 to 3-meter distance in around 2.9 seconds.

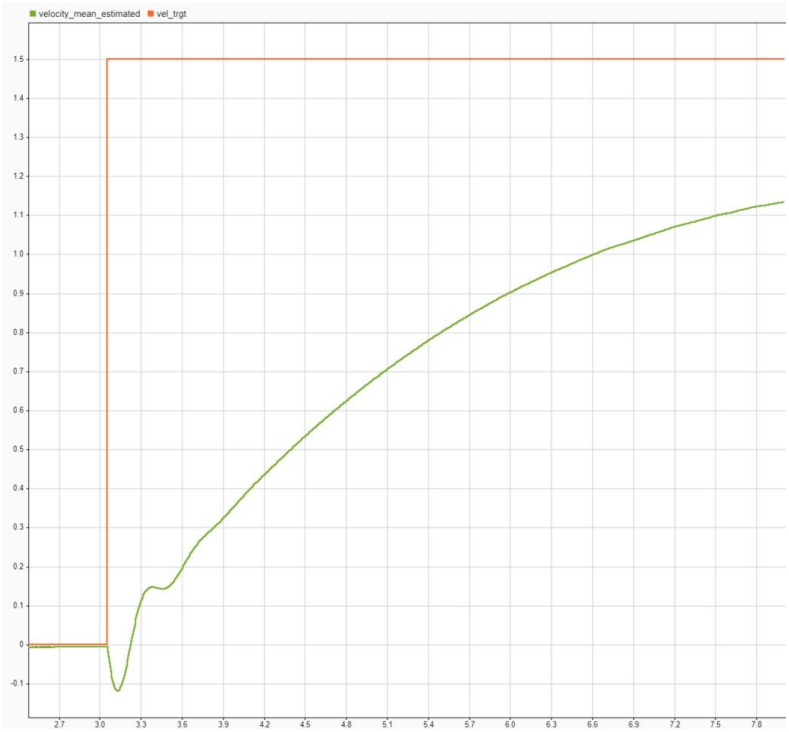

Figure 37 A time-plot of the 'vel_trg' (orange line) and the linear velocity of the virtual robot (green line)

7.4 Ramp function

In this section, the velocity target is substituted with a ramp input to analyze the system's response to a linearly increasing signal and assess the steady-state error. As expected, smoothing the trajectory of the input signal can lead to a faster system response. But a lot of difference was found between the physical robot and simulation virtual results , the simulation gave the same result for both velocity target (step input and ramp function), but in physical robot it was reduced the slight time from 3.2 seconds to 3 seconds.

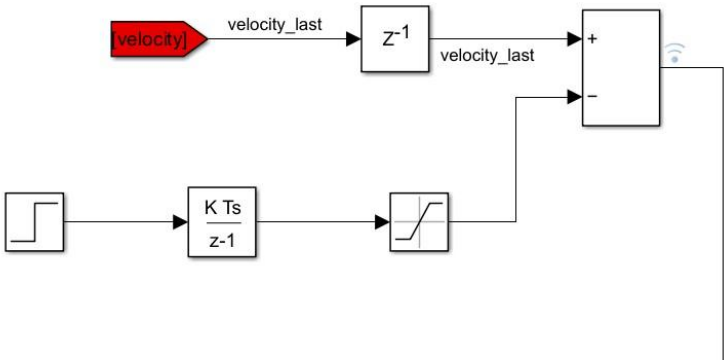

Figure 38 Ramp signal

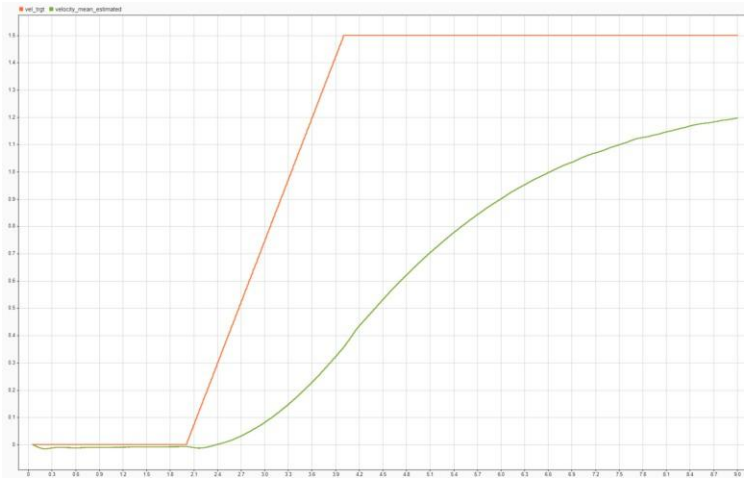

Figure 39 A time-plot of 'vel_trg' (orange) & linear velocity of virtual robot (green) with ramp input

7.5 Feed-forward controller

To minimize system response lag and reduce the workload on the feedback controller, a feedforward controller was integrated into the BLDC controller. This was achieved by incorporating two input signals: the velocity target and the controlled output signal. The key challenge in obtaining the desired outcome was ensuring the predictability of the robot's motion. The physical robot demonstrated improved stability with reduced oscillations and slightly better performance compared to previous results. For both step input and ramp input variations, the time required was 3.1 seconds without a run-up and 3 seconds with a run-up.

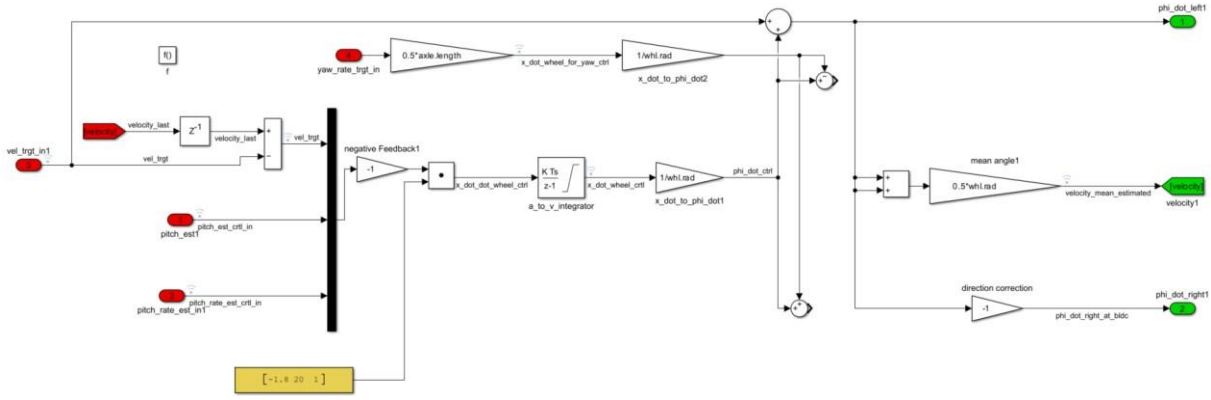

Figure 40 Feed-forward controller with feedback controller

7.6 PID Controller with a position target

A **PID controller** is a widely used control mechanism in industrial and robotic systems that ensures precise and stable operation by continuously adjusting the control input based on feedback from the system. It operates using three key components: Proportional (P), Integral (I), and Derivative (D) terms. The proportional term produces a correction that is directly proportional to the error (difference between the setpoint and the actual value). The integral term addresses the accumulation of past errors over time, ensuring the system eliminates

steady-state errors. The derivative term predicts future errors based on the rate of change of the error.

To achieve a faster rise time and an acceptable steady-state error in the robot's linear velocity, a position error signal was generated by subtracting the robot's actual position from the target position, which was input into the PID controller using Go-To and Form blocks. Additionally, the error signal passed through a sum block, where it was combined with the velocity target.

In Challenge 1, the robot was first stabilized by setting vel_trgt_auto and yaw_rate_trgt_auto to zero using the "after" command. In the subsequent state, position_trgt was set to 4 meters, allowing the robot sufficient time to reach its maximum velocity while ensuring its linear velocity decreased as it approached precisely 3 meters. By manually adjusting the proportional, integral, and derivative gains to 0.47, 0.1, and 0.01, respectively, the run-up time was reduced from 3 seconds to 2.8 seconds, achieving good stability and consistency. However, the simulation results consistently differed from the robot's actual testing outcomes.

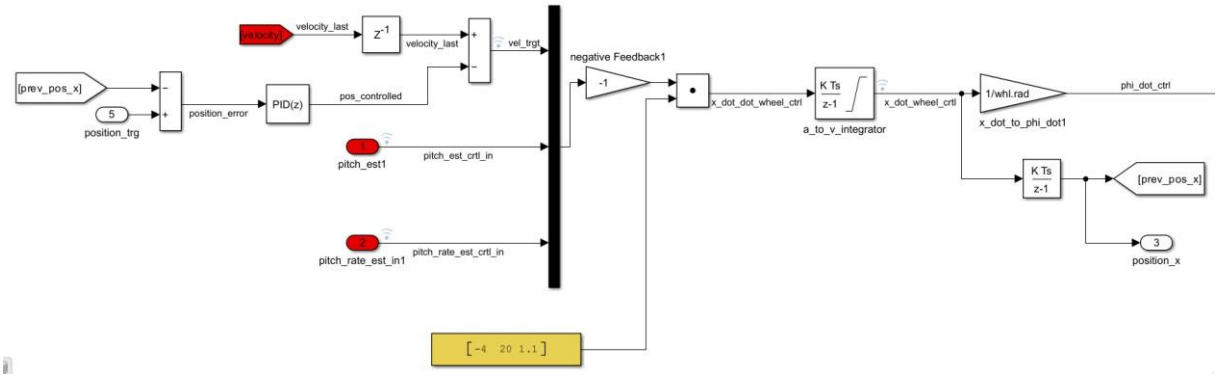

Figure 41 PID Controller with a position target

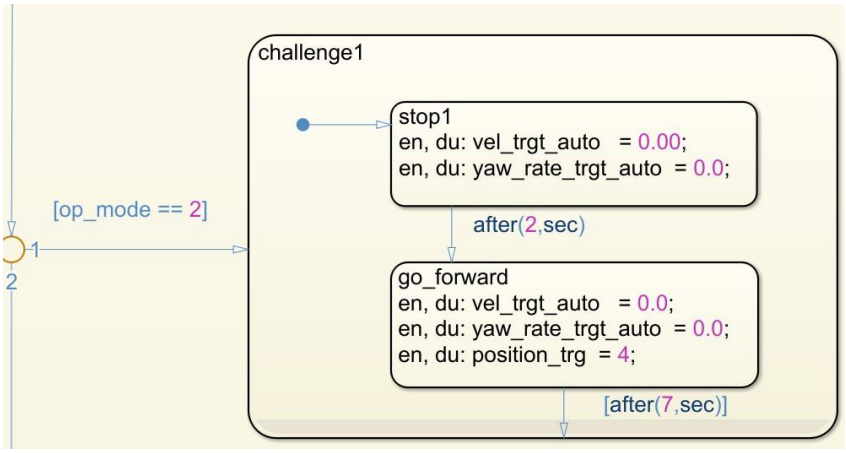

Figure 42 The state chart of the PID controller with position target implementation

7.7 Combining two solutions – PID Controller and state feedback controller

In the earlier steps, it was noted that the PID controller with a position target achieved faster acceleration without instability up to 1.5 meters. Beyond this point, significant oscillations were observed, prompting adjustments to the gains, which led to a reduction in linear

velocity. On the other hand, the state feedback controller demonstrated excellent acceleration and linear velocity performance for the remaining 1.5 meters. Consequently, the two controllers were combined to achieve an optimal solution for reaching the 3-meter finish line in the shortest possible time. The physical robot achieved an average time of 2.7 seconds, marking an improvement of 0.1 seconds.

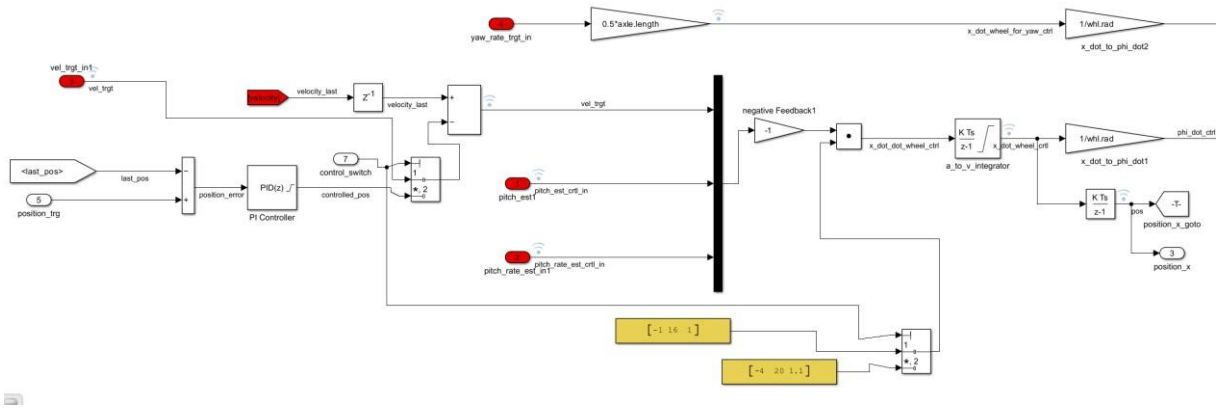

Figure 43 The combined PID and state feedback controller

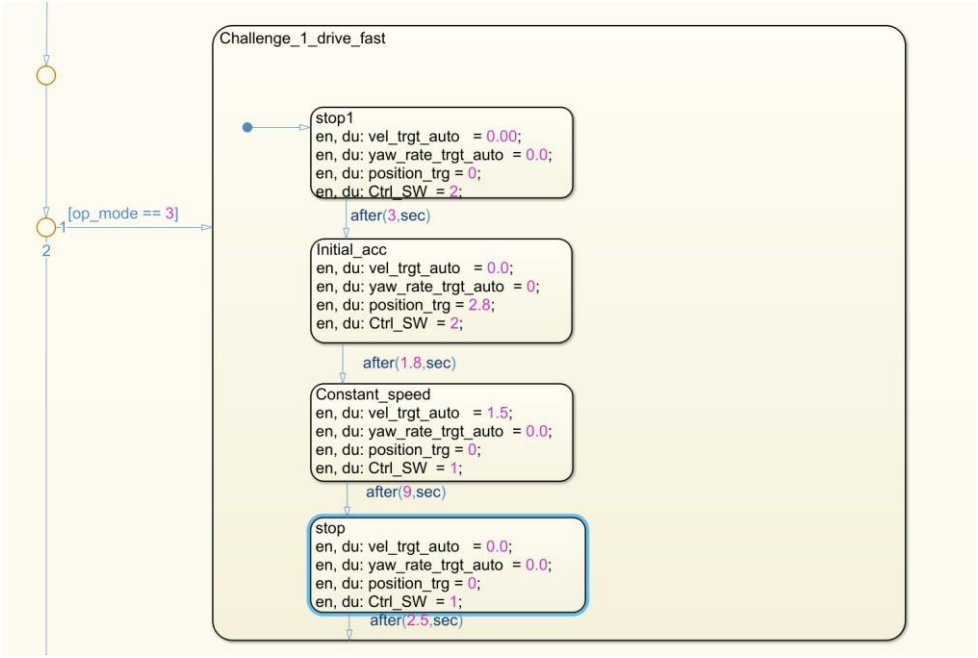

Figure 44 The state chart of the combined solution of PId and feedback controller

7.8 Overview of the different approaches for the challenge 1

S. No.	Controller Used	Virtual Robot Run-Time (seconds)	Physical Robot Run-Time

1	State feedback-Gains adjusted by hand	2.1	2.9
2	State feedback-Using ramp function	2.1	3
3	State feedback-With feed-forward	2	3
4	PID-With position target	1.4	2.8
5	State feedback and PID-Combined	1.3	2.7

8 Challenge2 – Driving Fast with two 360^0 Turns

8.1 Inconsistency of the AFTER command

This challenge emphasizes precision and accuracy in achieving the target within the specified maximum deviation. The robot must stop at 1 meter and perform a 360° rotation with a deviation limit of ±5 cm. Subsequently, it must move to the 2-meter mark, execute another 360° rotation, and remain within a deviation range of ±10 cm. To complete this task, the 'AFTER' command was used to operate the robot. However, its behavior was highly erratic during each transition. At times, the robot would rotate before or after reaching the 1-meter mark. Moreover, it failed to complete a full revolution, with rotations measuring approximately 340^0 and 350^0 degrees. Additionally, the error compounded at every 1-meter mark, causing the robot to completely deviate from the target line. Despite extensive experimentation with the temporal command, the results were only intermittently satisfactory, lacking consistency. Ultimately, it was determined that the task's performance was heavily dependent on three factors: the robot's position, yaw angle, and pitch angle.

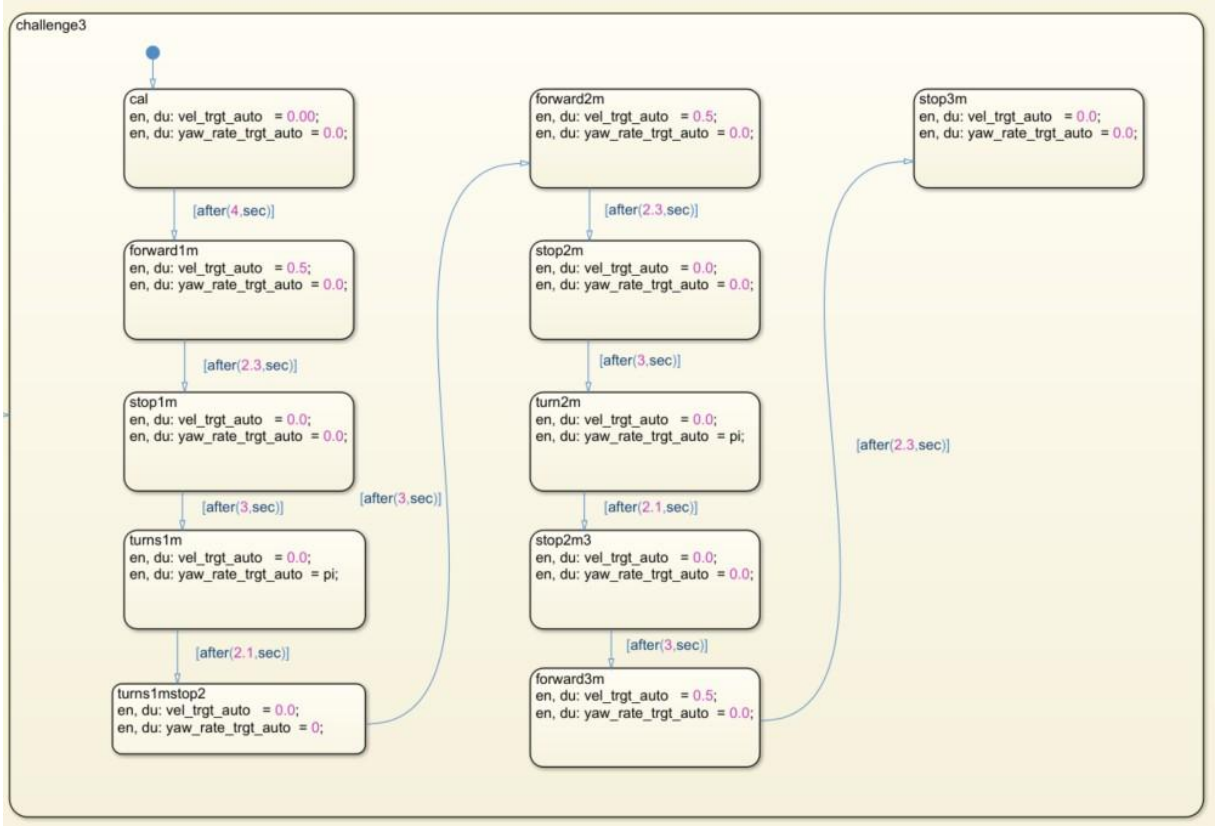

Figure 45 The state chart of challenge 2 using AFTER commands

8.2 Obtaining the yaw angle from IMU and the position from controller

Using the IMU, six signals were obtained: three pairs of linear accelerations from the accelerometer and three pairs of angular velocities from the gyroscope, corresponding to the X, Y, and Z axes. These angular velocities represent the roll rate, pitch rate, and yaw rate along the X, Y, and Z axes, respectively. The yaw rate signal was processed using a Simulink function (complementary filter), as illustrated in Fig. . The yaw angle was subsequently determined by integrating the yaw rate signal.

The robot's position along the X-axis was calculated by performing a double integration of the acceleration signal in the controller. By precisely controlling the robot's position, it successfully stopped at 1 m, 2 m, and 3 m, with minimal deviation from the target positions.

```
function [pit_est, pit_rate_est,yaw_rate] = estimate_angulars_script(acc_forward_meas, acc_vertical_meas, pitch_rate_meas, yaw_rate_raw, fc, dt)
persistent tau C pitch_from_acc_LP_filtered pitch_from_gyro_HP_filtered
if isempty(tau);
    tau = 1./ (2*pi*fc);
    C = tau/(tau+dt);
    pitch_from_acc_LP_filtered = 0.0;
    pitch_from_gyro_HP_filtered = 0.0;
end
pitch_from_acc = atan2(acc_forward_meas, acc_vertical_meas);
pit_rate_est = pitch_rate_meas*pi/18000; %result in [rad/s]
pitch_from_acc_LP_filtered  = C*pitch_from_acc_LP_filtered +(1-C)*pitch_from_acc;
pitch_from_gyro_HP_filtered = C*(pitch_from_gyro_HP_filtered+pit_rate_est*dt);
pit_est = pitch_from_acc_LP_filtered + pitch_from_gyro_HP_filtered; %result in [rad]
yaw_rate = yaw_rate_raw*pi/18000;
```

Figure 46 The yaw rate addition to the MATLAB code for the complementary filter

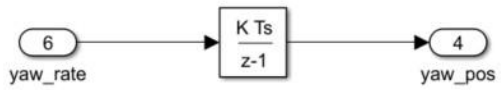

Figure 47 The calculation of yaw angle by integrating yaw rate

8.3 Implementation of PI controller

The PI controller was chosen for this challenge due to the instability and noise in the measurement signal caused by the derivative gain in the PID controller. Increasing the derivative gain resulted in a noisy measurement signal and system instability. Additionally, the need to minimize steady-state error, driven by the stringent accuracy requirements of this challenge, was a critical factor in the decision. PI controllers perform better at eliminating steady-state errors, offering superior accuracy.

Translation and rotational control are managed using position and yaw angle adjustments. To stop the robot at distances of 1 m, 2 m, and 3 m, the position was configured to match these fixed distances. As per the challenge requirement the position was fixed in the conditional command.

The yaw angle was configured to be greater than or equal to 2π (360°). For an additional 360° rotation, the yaw angle had to exceed or equal 4π. The system exhibited an acceptable positional deviation of approximately 1–2 cm, achieving precise position and yaw angle

control following the implementation of a PI controller with position and yaw adjustments. The proportional and integral gains used were set to 2 and 0.00001, respectively. However, the system encountered instances of undershoot and overshoot due to the need to balance the position error during the robot's rotation.

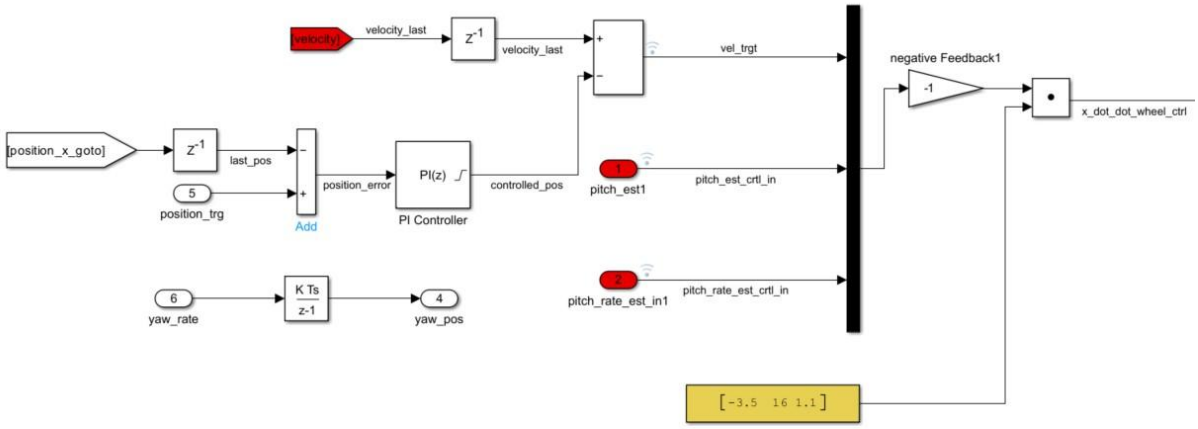

Figure 48 PI controller with position control

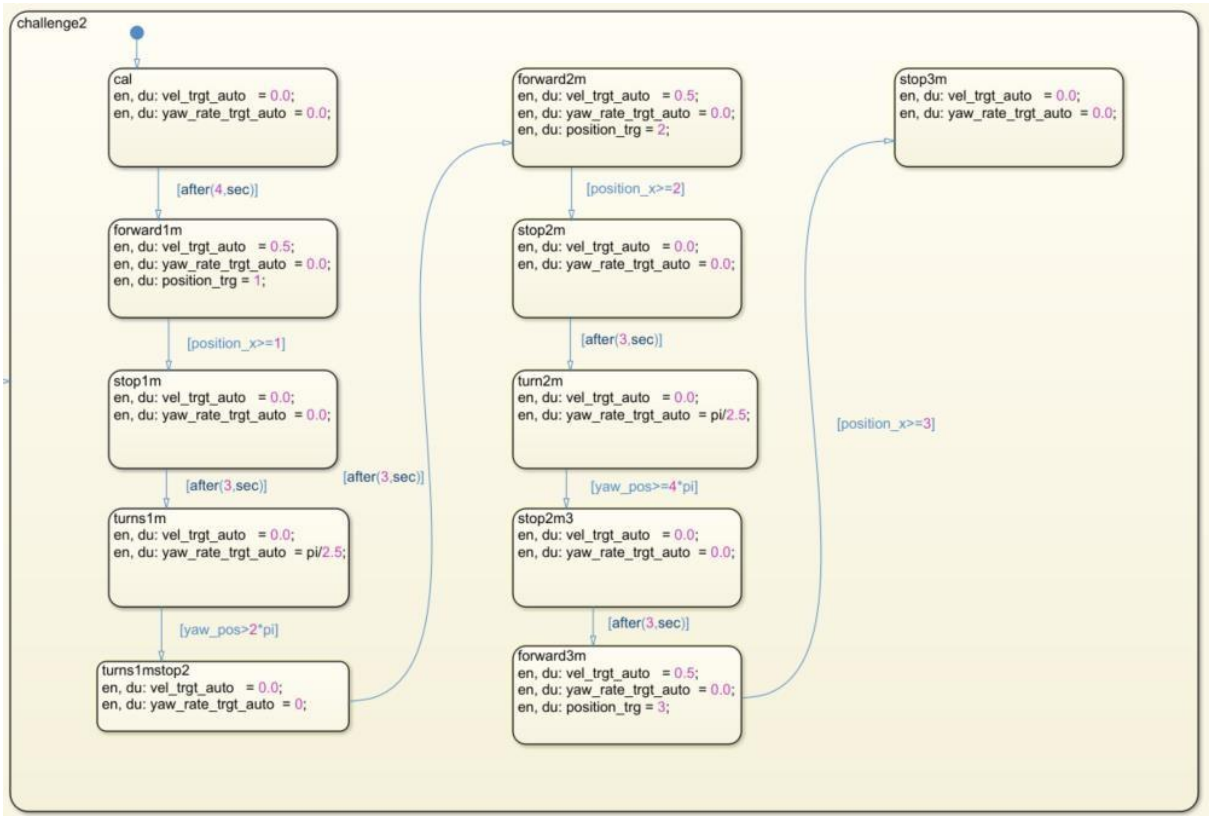

Figure 49 The state chart for PI controller

8.4 Optimised PI controller with target switching

The issue of undershooting and overshooting the rotation target was resolved by implementing a multiport switch. This switch functions by routing one of its input signals to the output, based on the value of a control signal. It alternates between the position target

and yaw target during linear and rotational acceleration. At any given moment, it engages with only one target. For instance, after 1 meter of translational movement, it switches to the yaw target, and vice versa. Significant performance improvements were observed, with minor rotational deviations of ±20 degrees and translational deviations of 1–2 cm. Furthermore, the P and I gains were maintained as in the previous step. In this challenge, adjusting temporal command and feedback gains only approximates the desired path, often leading to inconsistency and instability.

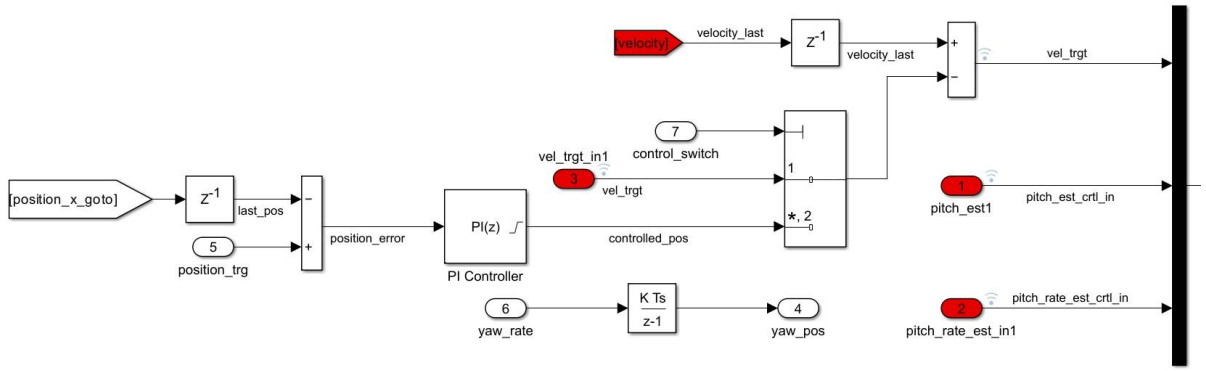

Figure 50 Multiport Switch with PI controller

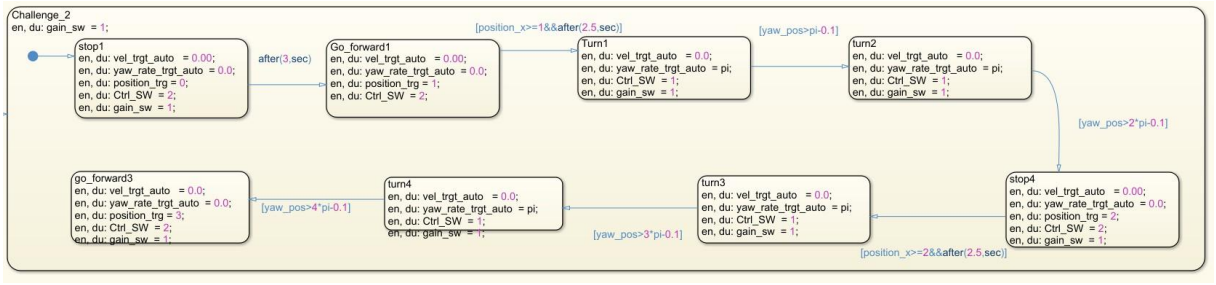

Figure 51 The state chart of the PI controller with a multiport switch

8.5 Summary of the task 2

S. No.	Solution Variants	Positional Deviation at 1 meter (cms)	Positional deviation at 2 meters (cms)	Positional deviation at 3 meters (cms)	Angular Deviation At 1 meter (x^o)	)Angular Deviation At 2 meters (x^0)
1	After commands	7	11	13	7	13
2	PI controller	2	3	4	15	18
3	PI with target switches	1	2	3	4	6

9 Challenge 3 – Compensate an additional weight

9.1 Implementation of the PID controller

In this challenge, the robot must balance the additional weight with minimal deviation and return to its zero position within the shortest possible settling time. The primary focus is on position and accuracy control. The derivative gain anticipates future errors by analysing the rate of error change, functioning as a damper to enhance system stability, reduce overshoot, and shorten settling time. When a weight exceeding 570 g was applied, the system exhibited unstable oscillations, which intensified as the weight increased The robot demonstrated stable performance for weights up to 570 g. The maximum deviation from initial position 52 cm in both directions, it took 14 and 6 second to reach at zero position for the weights 250 g and 320 g (250 + 320 = 570) respectively.

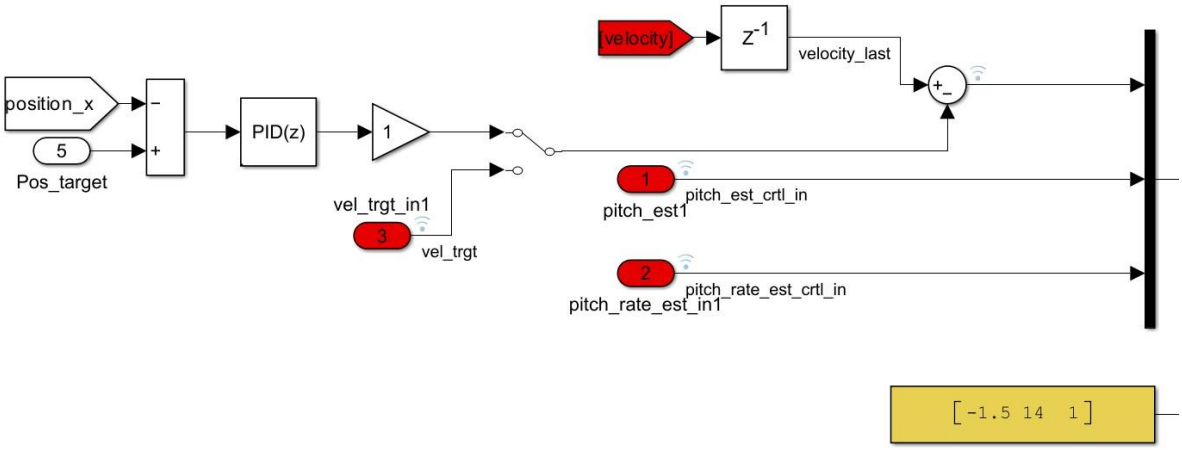

Figure 52 The controller for challenge 3 with a PID block

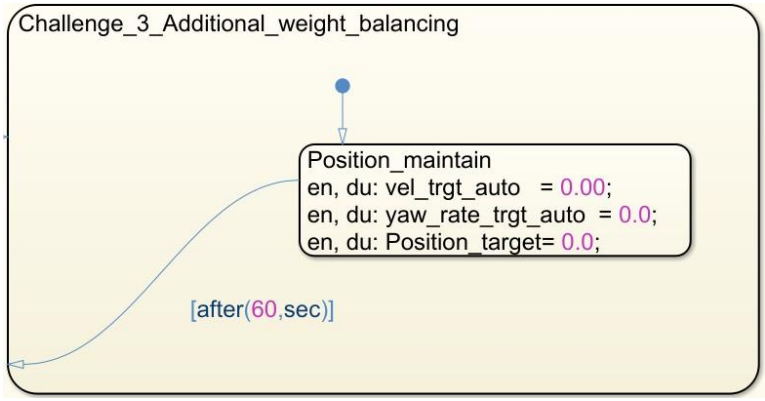

Figure 53 The state chart for challenge 3

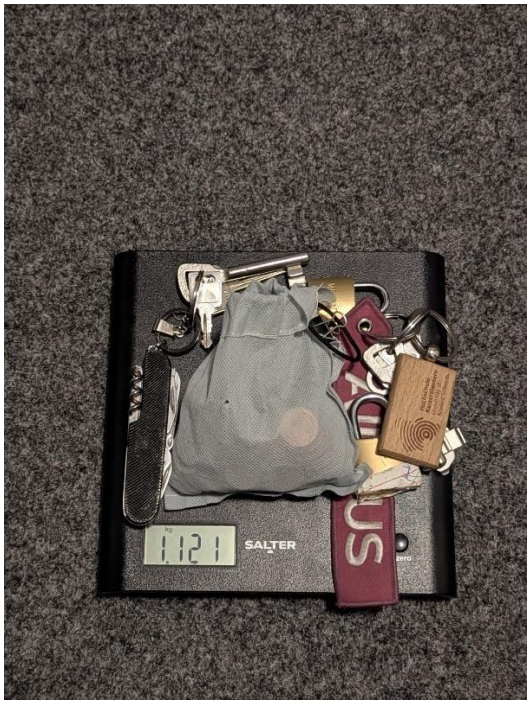

Figure 54 Total applied weight

9.2 Cascaded PID controller Implementation

To address the issue in the previous attempt, an additional PID controller was implemented to manage the velocity error signal. This resulted in significant improvements in the robot's performance, including reduced settling time and deviation. The physical robot returned to its initial position of 0 in 15 seconds after the first weight(180 g) was added, 20 seconds after the second weight(620 g), and 3 seconds after the third weight(220 g), then in 2 seconds after the fourth weight of 50g, and in 2 seconds after adding the final weight of 50g after all weights were added. The PID gains were fine-tuned through a trial-and-error approach to achieve the desired stability and consistency in the solution.

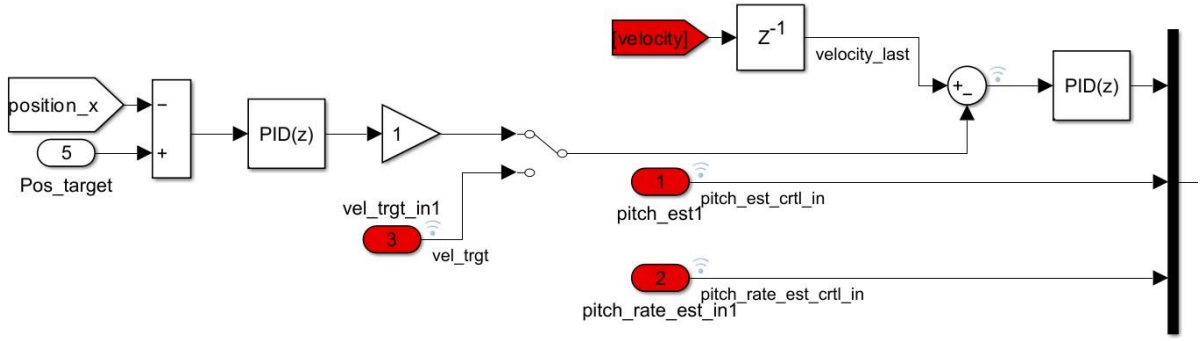

Figure 55 The cascaded PID controller implementation for challenge 3

S. No.	CONTROLLER USED	Weight Used (g)	Settling time (sec)	Maximum Deviation (cm)	Steady-state Error (cms)
1	PID controller	250 +320	14 and 6	52	4
2	Cascaded PID controller	1120	15, 20, 3, 2.2	36	0-3

10 Challenge 4 – Driving a figure 8

10.1 Rotation control by electrical angles

To control the linear and angular velocity of the Robot precisely, a new idea was applied to trace 8 – like figures. By controlling electrical angle and yaw rate the robot was traced the path approximately by changing the values of electrical angle by hands. The electrical angle was evaluated from the mechanical angle.

$$Electrical\ angle = Number\ of\ pole\ pairs \times Mechanical\ angle$$

7 pole pair was used in the BLDC motor, for every 1^0 of mechanical rotation, the electrical angle changes by 7^0. A rough calculation was done to determine the number of electrical angle for semicircle with radius 1 m. The task was divided into three parts ,a semi-circle , full circle and a semicircle again.

$$Number\ of\ rotation\ to\ drive\ half circumference = \frac{Half\ circumference\ of\ target\ circle}{Circumference\ of\ the\ Wheel}$$

$$Number\ of\ rotation\ to\ drive\ half\ Circumference = \frac{1.57}{0.2827}$$

$$Number\ of\ rotation\ required\ to\ drive\ half\ circumference = 5.55\ rotations$$

$$1^0\ mechanical\ angle\ of\ rotor = 7^0\ electrical\ angle$$

$$360^0\ mechanical\ angle\ of\ rotor \times 5.55\ rotations = 7^0\ electrical\ angle \times 360^0 \times 5.55\ rotations$$

$$1998^0\ mecahnical\ angle\ of\ rotor = 13986^0\ electrical\ angle$$

$$34.87\ radians\ mechanical\ angle\ of\ rotor = 244.1\ radians\ electrical\ angle$$

$$Target\ electrical\ angles\ for\ driving\ half\ circumference = 244.1\ radians\ electrical\ angle$$

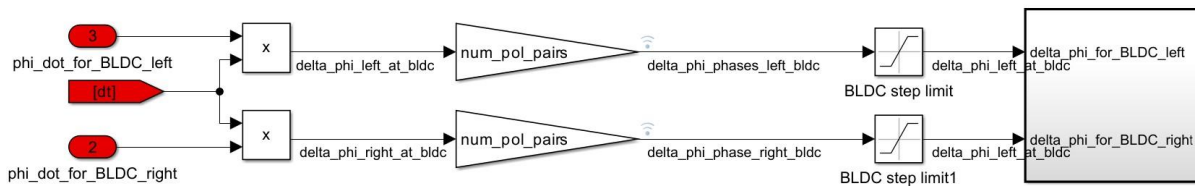

Figure 56 Convert mechanical and electrical angle in the BLDC driver

The cumulative electrical angle signal is provided to the autonomous targets state chart by establishing output ports in the BLDC driver subsystem and corresponding inputs in the state chart. The electrical angles of the left and right wheels are utilized to manage the robot's turning, leveraging the differential action of the robot. Small correction factors are incorporated into the transition signals since the theoretically calculated values do not directly apply in practical scenarios due to factors such as surface friction, wheel oscillations for self-balancing, and other minor influences. After spending lot time with this idea we did'nt get any optimal solution to track the path exactly , some time it goes under circle and over circle. Moreover, it calculate the electrical angle when the robot try to balance itself.

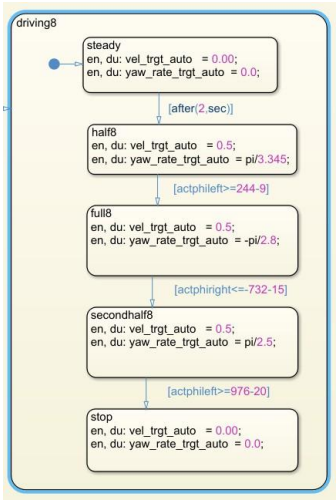

Figure 57 The state chart of the robot controlled using electrical angles

10.2 Yaw rate calculation

The yaw rate is obtained by differentiating the yaw angle with respect time. This variable was further used in the autonomous targets charts.

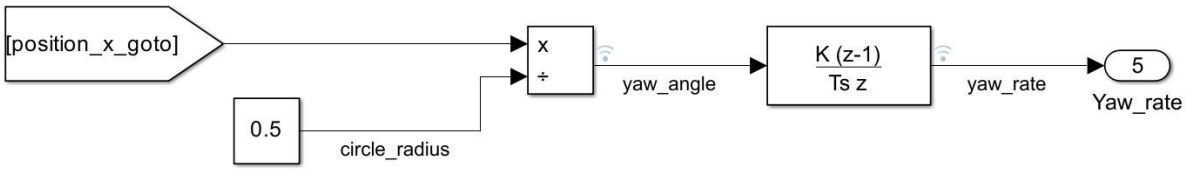

Figure 58 Yaw rate calculation

10.3 Autonomous target

To create an 8-figure trajectory, either two full circles or a combination of semicircles was required. An autonomous target chart consisting of seven states was developed. The circumference of a half-circle is 1.57, so the position target was incrementally set at 1.57 for each step. The calculated yaw rate was incorporated into the state chart as the yaw rate target to guide the robot along the desired path. Following this state chart, the physical

robot first drives a semicircle, completes the circle with another semicircle, then drives two quarter-circles, and finally completes the run with a semicircle.

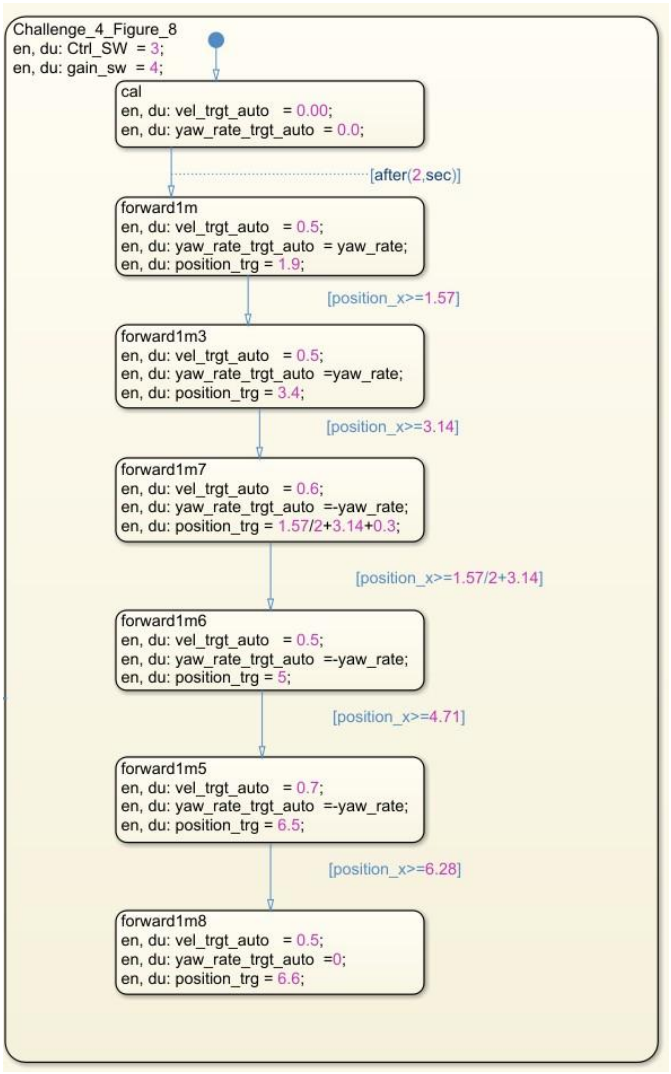

Figure 59 The state chart for the challenge 4

10.4 Summary of challenge 4

S. No.	Task 4 all solution	Track follow %age	Time Duration(sec)
1	PI controller – Electrical angle controlled	80 %	14
2	Autonomous Target	100 %	8.4

11 DIP switches

To deploy all challenges, the robot is equipped with four DIP switches. Each challenge can be assigned to an individual switch. These signals are processed through their respective gain blocks, each with a gain of 5/1023. This results in an output of 0 for an inactive DIP switch and 5 for an active one. The gain block outputs are then connected to a truth table. Based

on the received inputs, the truth table determines the operation mode. A valid operation mode is outputted only when a single DIP switch is active. If multiple switches are active or none are active, the system defaults to the balancing mode.

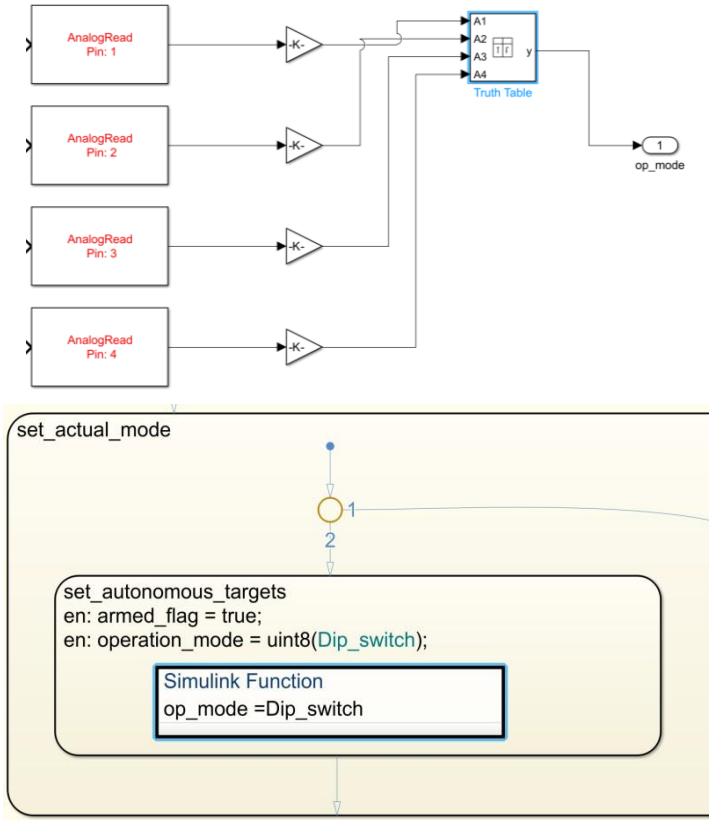

Figure 60 The state chart for DIP switches

Condition Table

	DESCRIPTION	CONDITION	D1	D2	D3	D4	D5
1	Dip_Switch_1	A1>=4.9	T	F	F	F	-
2	Dip_Switch_2	A2>=4.9	F	T	F	F	-
3	Dip_Switch_3	A3>=4.9	F	F	T	F	-
4	Dip_Switch_4	A4>= 4.9	F	F	F	T	-
		ACTIONS: SPECIFY A ROW FROM THE ACTION TABLE	1	2	3	4	5

Action Table

	DESCRIPTION	ACTION
1	Challenge_1	y = 2;
2	Challenge_2	y=3
3	Challenge_3	y=4
4	Challenge_4	y = 5;

Figure 61 Truth Table to switch from one challenge to other

12 References

1. Bitsch, G., System level rapid development in mechatronics WS 2024/25, Hochschule Kaiserslautern, Kaiserslautern, 2024.

2. Hanselman, D., Brushless Permanent Magnet Motor Design, 2nd edition, Magna Physics Publishing, Ohio, 2006

3. Aström, K.J., and Murray, R.M., Feedback Systems, Princeton University Press, New Jersey, 2009

4. MATLAB Documentation, The MathWorks Inc., 2024.

5. Nise, N.S., Control Systems Engineering, 7th edition, John Wiley & Sons Inc., 2015, Hoboken, 2015